Nano-Task Decomposition & Execution Engine

Complete Technical Specification — Direct Implementation Guide

Revision: 1 **Last modified:** 2026-07-24T00:00:00Z **Description:** Complete technical specification for the Nano-Task Decomposition & Execution Engine — the core innovation of the Helix Constitution–Powered SpecKit–Superpowers Bridge. Decomposes SpecKit-generated tasks into the smallest possible, fully decoupled, self-contained nano-tasks. **Authority:** Constitution §11.4.102 / §11.4.27 / §11.4.142 / §11.4.145 / §11.4.194 **Maintainer:** HelixDevelopment **Scope:** Universal (§11.4.17) — consumed by reference from the constitution submodule; every project-supplied config is DATA per §11.4.35.[1]

---

[1] [1] . Concept

. The Problem: Weak Local LLMs

The Helix Constitution–Powered SpecKit–Superpowers Bridge targets local, self-hosted LLM inference — defaulting to a workstation-grade host (AMD Threadripper 64-core, 256 GB DDR5, RTX 32 GB VRAM). Local models are unavoidably **weaker** than frontier cloud models on any single given task: context windows are smaller, reasoning depth is shallower, and instruction-following precision is lower. A [1] [2] monolithic "implement this feature" task that a cloud model can handle in one shot will overwhelm or produce low-quality output on a local model.

. The Solution: Nano-Task Decomposition

The nano-task engine decomposes every spec-kit-generated task into the **smallest possible, fully decoupled, self-contained atomic units** — nano-tasks. Each nano-task is bounded to ≤512 effective tokens of context (the implementation

surface the LLM must reason about), has an explicit typed input/output contract, a list of zero or more dependency nano-task ids, and a RED-first test template that must FAIL before implementation begins.

Three principles drive the design:

1. **Maximum decoupling.** A nano-task must be implementable with zero knowledge of any other nano-task's internals. It only knows its upstream dependencies by their output schemas. Two nano-tasks at the same topological level are fully parallel.
2. **Layered composition builds complexity incrementally.** Nano-tasks that produce raw primitives are consumed by nano-tasks that compose those primitives, which are consumed by **binder tasks** (the parent task that groups them). Only the binder validates integration — nano-tasks validate only their own contract.
3. **TDD at the nano level.** Every nano-task ships a RED test (must fail before implementation) and passes GREEN only when the implementation satisfies the output contract. No nano-task is "done" without captured evidence per §11.4.5 / §11.4.69.

[1] [3]

. The Nano-Task as a Universal Work Unit

A nano-task is the **atomic unit of work** in the SpecKit→Superpowers pipeline. It is:

- **Small enough** that a modest local LLM (7B–13B parameter class, 4-bit quantized) can execute it flawlessly in a single inference pass.
- **Self-contained enough** that its implementation only reads its typed input, its dependency outputs, and its test template.
- **Testable in isolation** — the RED→GREEN TDD cycle runs against the nano-task alone, with upstream dependencies satisfied by their verified output artifacts (never mocks).

---

2 1

. Nano-Task Schema

. Complete YAML Specification

```yaml
# ── Nano-Task Canonical Schema ──────────────────────────────────────
# Every nano-task is a single YAML document.  This schema is THE contract
# the decomposer writes and the executor reads.  No field is optional.

# ── Identity ────────────────────────────────────────────────────────

id: "nano-001"                  # Unique within the workspace; kebab or
                                # dot-separated hierarchical (e.g.
                                # "auth.bcrypt.hash-password").

parent: "binder-auth"           # The parent binder task id.  A nano-task
                                # always belongs to exactly one binder.

# ── Human-readable metadata ─────────────────────────────────────────

title: "Hash plaintext password with bcrypt"
                                # One sentence — what this nano-task does.

description: |                  # 4–8 sentence plain-language description
  Accept a plaintext password string and a cost factor.  Hash the password
  using bcrypt with the given cost.  Return the hashed password as a
  hex-encoded string.  This nano-task does NOT validate the password length
  or strength — that is a separate upstream nano-task.

type: "function"                # Closed set:
                                #   function  — pure computation (no I/O)
                                #   integration — external API / DB / FS
                                #   stateful  — mutates shared state
                                #   binding   — composes other nano-tasks
                                #   test-only  — test harness / probe

# ── Input / Output Contracts ────────────────────────────────────────

input:
  schema:                       # JSON Schema (draft-2020-12 subset)
    type: object
    properties:
      password:
        type: string
        minLength: 1
        description: "Plaintext password to hash."
      cost:
        type: integer
        minimum: 4
        maximum: 31
```

```
      default: 12
      description: "bcrypt cost factor."
  required: ["password"]
 example:                    # At least one concrete valid example
  password: "s3cret!"
  cost: 12

output:
 schema:
  type: object
  properties:
   hash:
    type: string
    pattern: "^\\$2[aby]\\$\\d{2}\\$[./A-Za-z0-9]{53}$"
    description: "Bcrypt hash string (modular crypt format)."
  required: ["hash"]
 example:
  hash: "$2b$12$LJ3m4ys3GZqZqZqZqZqZqOqZqZqZqZqZqZqZqZqZqZqZq"

# ── Dependency Graph ──────────────────────────────────────────────

dependencies: ["nano-000"]        # Ordered list of nano-task ids that
                                  # MUST complete before this one starts.
                                  # Empty list [] = root nano-task.
                                  # The output of each dependency is
                                  # available at runtime via its id.

# ── Token Budget ──────────────────────────────────────────────────

estimated_tokens: 384             # Decomposer's estimate of the effective
                                  # implementation-surface token count.
                                  # MUST be ≤ 512.  The executor validates
                                  # this post-implementation and flags
                                  # violations for re-decomposition.

# ── Test Template (RED-first) ─────────────────────────────────────

test_template: |
 # RED: must FAIL before implementation exists
 # §11.4.115 — RED-baseline-on-the-broken-artifact
 # §11.4.43  — TDD-Fix-Discipline step 1

 import { executeNanoTask } from "../runtime/executor";
 import { hashPassword } from "./impl";   // WILL NOT EXIST yet

 const result = executeNanoTask("nano-001", {
```

```
  password: "test-password-123",
  cost: 4
});

// Assert output shape
assert(result.hash, "hash must be present");
assert(typeof result.hash === "string", "hash must be a string");
assert(result.hash.startsWith("$2"), "must be bcrypt modular crypt format");

// Assert the hash is deterministic per cost + salt and
// verifies against the input password
const verifyResult = executeNanoTask("nano-003", {
  password: "test-password-123",
  hash: result.hash
});
assert(verifyResult.valid === true,
  "hash must verify against the original password");

// GREEN: must PASS after implementation lands
// §11.4.115 polarity switch RED_MODE=0
```

```
test_evidence_class: "runtime"       # §11.4.226 evidence class:
                                     #   runtime  — target-executed observable
                                     #   artifact — path + content hash
                                     #   source   — grep/static analysis
                                     # Must match the defect layer's floor.

# ── Implementation Hints ──────────────────────────────────────────────

implementation_hints:
  - "Use bcrypt with cost factor 12 as the default when cost is not provided."
  - "Return the hashed password as a hex string using the modular crypt format."
  - "DO NOT store or log the plaintext password — return only the hash."
  - "The bcrypt library MUST be the native binding, not a pure-JS fallback."
  - "Reject passwords longer than 72 bytes (bcrypt input limit) with a typed error."
  - "The cost factor MUST be clamped to [4, 31] inclusive; out-of-range → error."

# ── Acceptance Criteria ───────────────────────────────────────────────

acceptance:
  - "RED test fails before implementation (expected: impl module not found)."
  - "GREEN test passes with cost=4, password='test-123'."
  - "GREEN test passes with cost=12 (default), password='test-123'."
  - "Different passwords produce different hashes (non-determinism proof)."
  - "Cost=31 produces a hash (extreme edge)."
  - "Password > 72 bytes returns a typed ValidationError."
  - "Cost=3 returns a typed ValidationError (below minimum)."
```

```yaml
  - "Cost=32 returns a typed ValidationError (above maximum)."

# ── Workable Items Mapping ──────────────────────────────────────────
workable_item:                  # §11.4.93 / §11.4.54
  atm_id: "ATM-001"               # Stable, auto-incremented ticket id
  status: "Queued"                # §11.4.15 lifecycle status
  type: "Task"                  # §11.4.16 item type
  severity: "normal"
  assigned_to: "subagent-003"

# ── Execution Metadata ──────────────────────────────────────────────
execution:
  retry_policy:
    max_attempts: 3
    backoff: "exponential"          # exponential | linear | fixed
    backoff_base_ms: 1000
  timeout_ms: 300000              # 5 minutes — per-nano-task hard timeout
  parallel_group: null            # Set by the DAG engine at schedule time
                                  # (topological level index)
  requires_gpu: false             # True only for CUDA-accelerated nano-tasks
  min_memory_mb: 128              # Minimum RAM required for the worker

# ── File-Scope Manifest ─────────────────────────────────────────────
file_scope:                     # §11.4.58 PWU file-scope
  - "src/auth/hash_password.ts"
  - "tests/auth/hash_password.test.ts"
  - "schemas/auth/hash_password.schema.json"
```

- Field Reference

Field	Type	Required	Description
`id`	string	yes	Unique nano-task id within the workspace
`parent`	string	yes	Parent binder task id
`title`	string	yes	One-sentence human-readable summary
`description`	string	yes	4–8 sentence plain-language description
`type`	enum	yes	`function` / `integration` / `stateful` / `binding` / `test-only`
`input.schema`	object	yes	JSON Schema for input validation
`input.example`	object	yes	At least one valid concrete example
`output.schema`	object	yes	JSON Schema for output validation
`output.example`	object	yes	At least one valid concrete example
`dependencies`	string[]	yes	Ordered nano-task ids (empty = root)
`estimated_tokens`	integer	yes	Decomposer's token estimate (≤512)
`test_template`	string	yes	RED-first test code
`test_evidence_class`	enum	yes	`runtime` / `artifact` / `source`
`implementation_hints`	string[]	yes	≥2 concrete implementation hints

Field	Type	Required	Description
`acceptance`	string[]	yes	≥3 enumerated acceptance criteria
`workable_item.atm_id`	string	yes	Stable ATM-NNN ticket id
`workable_item.status`	string	yes	§11.4.15 lifecycle status
`workable_item.type`	string	yes	§11.4.16 item type
`execution.retry_policy`	object	yes	Max attempts + backoff strategy
`execution.timeout_ms`	integer	yes	Per-task hard timeout
`execution.parallel_group`	integer?	no	Set by DAG engine at schedule time
`execution.requires_gpu`	boolean	yes	CUDA-acceleration required?
`execution.min_memory_mb`	integer	yes	Minimum RAM for the worker
`file_scope`	string[]	yes	§11.4.58 PWU file-scope manifest

3

3 1

. Parent-Binder Architecture

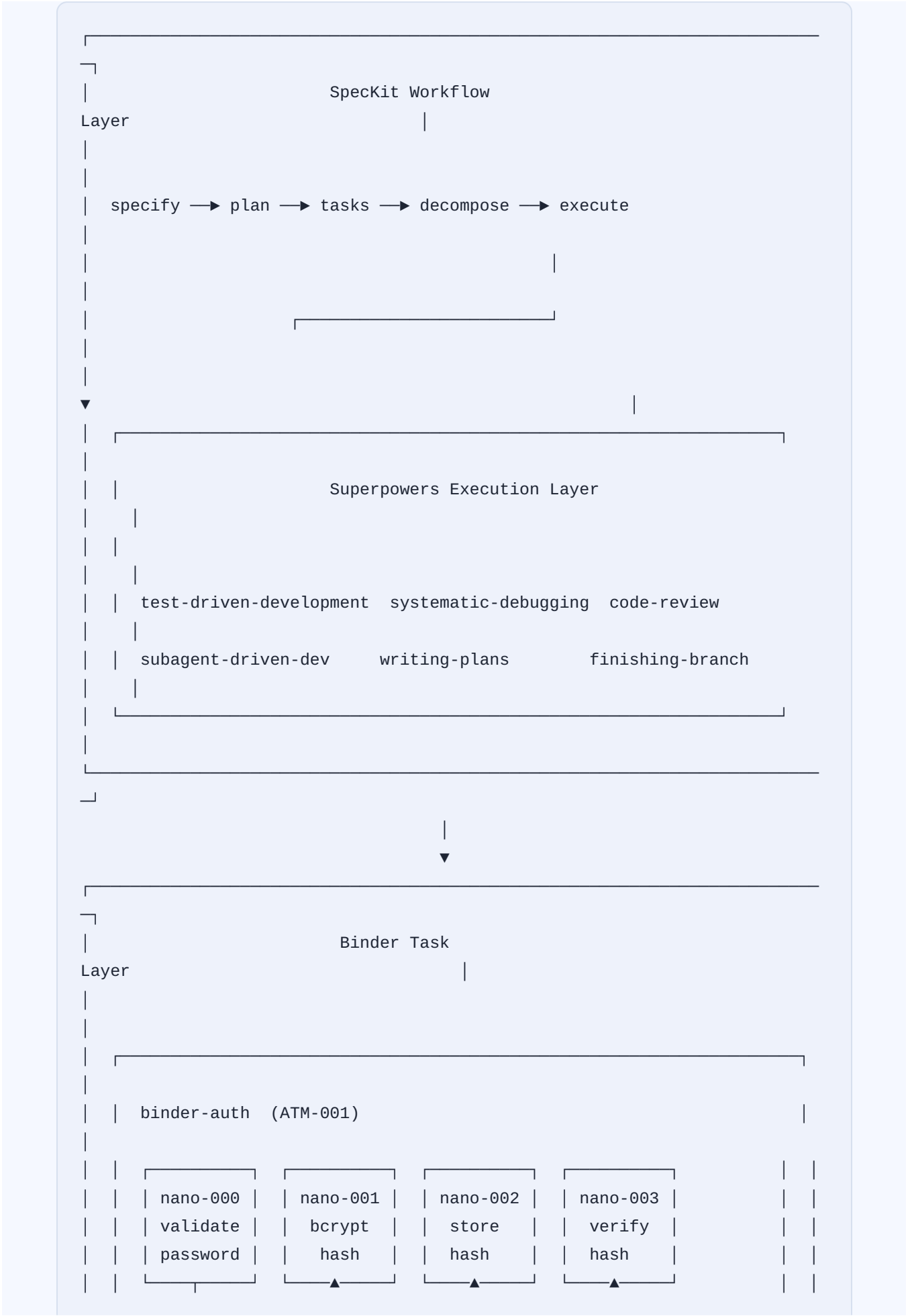
. Concept

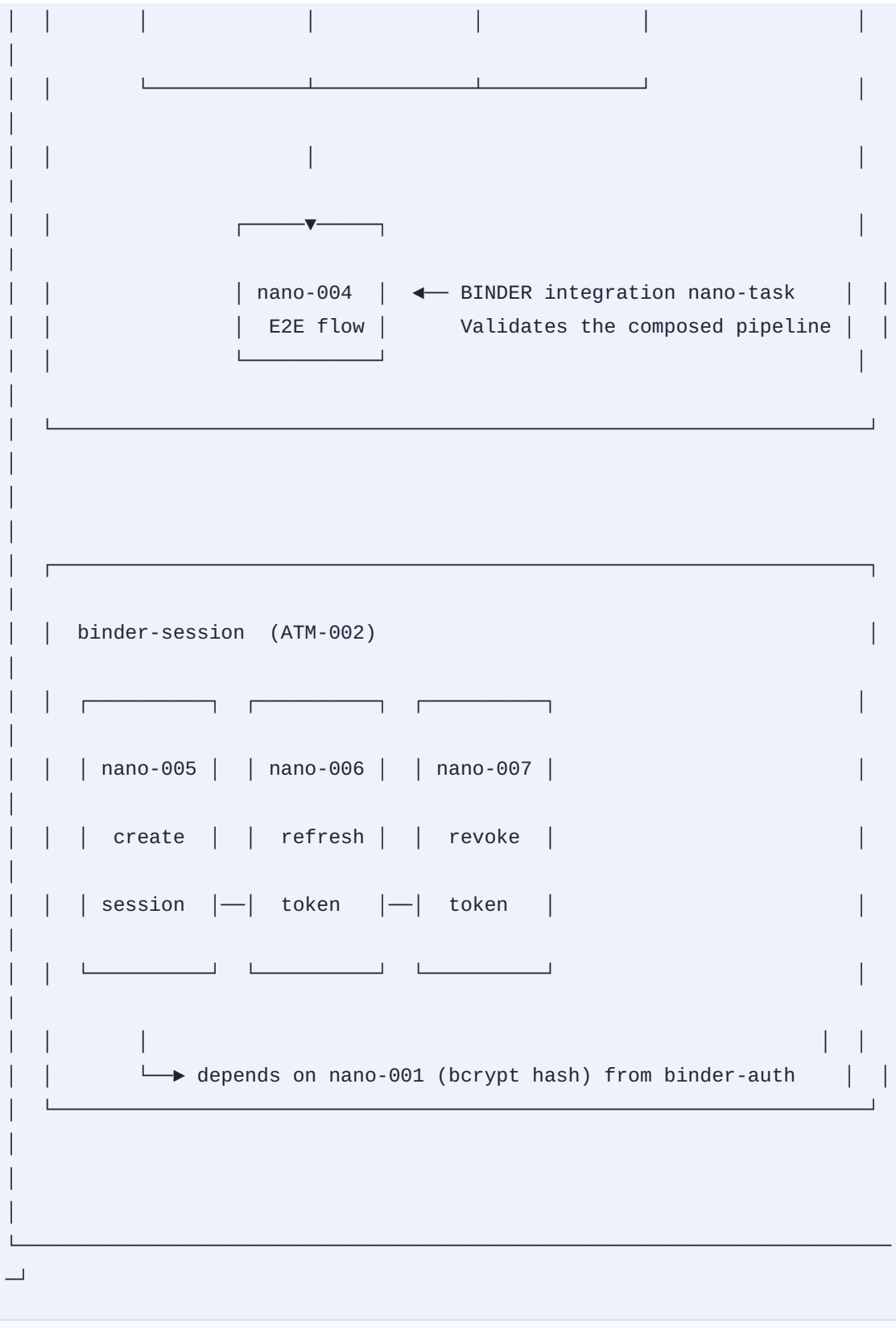
A **binder task** is the parent grouping a set of nano-tasks that together implement one SpecKit task. Binders:

- Define the shared namespace for their child nano-tasks.
- Own the **integration test** — the ONLY test that validates that nano-task outputs compose correctly.

- Define the **binder-level RED test** (fails before any child nano-task is implemented, passes GREEN only after ALL children pass GREEN and the integration test passes).
- Have their own workable item (ATM-NNN) per §11.4.93, with nano-tasks as sub-sub-items.

- Hierarchy Diagram


SpecKit Workflow Layer
specify → plan → tasks → decompose → execute
Superpowers Execution Layer
test-driven-development
systematic-debugging
code-review
subagent-driven-dev
writing-plans
finishing-branch
Binder Task Layer
binder-auth (ATM-001)
nano-000 validate password
nano-001 bcrypt hash
nano-002 store hash
nano-003 verify hash


nano-004
E2E flow
BINDER integration nano-task
Validates the composed pipeline
binder-session (ATM-002)
nano-005
create
session
nano-006
refresh
token
nano-007
revoke
token
depends on nano-001 (bcrypt hash) from binder-auth

. Binder Schema

```yaml
# binder-auth.yaml
id: "binder-auth"
title: "Authentication Subsystem"
description: |
  Complete authentication subsystem: password validation, bcrypt hashing,
  hash storage, and hash verification.  All four nano-tasks compose into
  a password-based auth flow.
spec_kit_task_ref: "tasks.md#user-auth"
status: "In progress"         # §11.4.15
type: "Feature"               # §11.4.16
atm_id: "ATM-001"

nano_tasks:
  - nano-000   # validate password strength
  - nano-001   # bcrypt hash
  - nano-002   # store hash in DB
  - nano-003   # verify hash against DB
  - nano-004   # E2E binding test (composes nano-000 → nano-003)

# The binder-level RED test: fails if ANY child nano-task is missing
# or if the composed pipeline does not work end-to-end.
binder_test: |
  # RED: must fail before ANY child nano-task is implemented
  # GREEN: must pass ONLY after ALL children are GREEN
  import { executeBinder } from "../runtime/binder";

  const result = executeBinder("binder-auth", {
    password: "Str0ng!Pass123",
    cost: 12
  });
3 4 assert(result.hashed, "pipeline must produce a hash");
  assert(result.verified, "pipeline must verify the hash");
```

. Cross-Binder Dependencies

Nano-tasks in one binder MAY depend on nano-tasks in another binder. The DAG engine resolves these transparently. Example: `nano-005` (create session, in `binder-session`) depends on `nano-001` (bcrypt hash, in `binder-auth`). The dependency is declared by id, and the executor resolves the output artifact regardless of which binder owns it.

4

4 1

. Dependency Graph (DAG) Engine

. Core Algorithms

The DAG engine is responsible for:

1. **Topological sort** — produce a linear order respecting all dependencies (Kahn's algorithm).
2. **Cycle detection** — reject any graph containing a cycle (a cycle means the decomposition is ill-formed).
3. **Parallel execution groups** — group nano-tasks by topological level (all tasks at level 0 run in parallel, then level 1, etc.).
4. **Incremental update** — when one nano-task completes or a new nano-task is added, re-compute only the affected sub-graph.

. Kahn's Algorithm

```
Algorithm: topologicalSort(nodes, edges)
Input:   nodes  — set of nano-task ids
         edges  — map of id → [dependency ids]
Output:  sorted — topologically sorted list
         OR     — error if a cycle is detected

1.  in_degree ← map of id → 0
2.  for each (id, deps) in edges:
3.      in_degree[id] ← len(deps)   // number of unfinished dependencies
4.
5.  queue ← []                       // nodes with in_degree = 0
6.  for each id in nodes:
7.      if in_degree[id] = 0:
8.          queue.push(id)
9.
10. sorted ← []
11. while queue is not empty:
12.     current ← queue.pop()
13.     sorted.push(current)
14.     for each (id, deps) in edges:
15.         if current in deps:       // current is a dep of id
16.             in_degree[id] ← in_degree[id] - 1
17.             if in_degree[id] = 0:
18.                 queue.push(id)
19.
20. if len(sorted) ≠ len(nodes):
21.     return ERROR: cycle detected
22. return sorted
```

. Parallel Execution Groups

After topological sort, group consecutive nodes that share the same dependency depth (distance from root) into **parallel execution groups**. All nano-tasks in group G have their dependencies satisfied by groups 0..G-1.

```
Algorithm: partitionIntoGroups(sorted, edges)
Input:   sorted — topologically sorted list from Kahn's algorithm
         edges  — map of id → [dependency ids]
Output:  groups — list of lists, each sublist is a parallel execution
group

1.  depth ← map of id → 0          // maximum dependency chain length
2.  for each id in sorted:
3.      if edges[id] is empty:
4.          depth[id] ← 0
5.      else:
6.          max_dep_depth ← max(depth[d] for d in edges[id])
7.          depth[id] ← max_dep_depth + 1
8.
9.  groups ← []
10. current_depth ← 0
11. current_group ← []
12. for each id in sorted:
13.     if depth[id] > current_depth:
14.         groups.push(current_group)
15.         current_group ← [id]
16.         current_depth ← depth[id]
17.     else:
18.         current_group.push(id)
19. groups.push(current_group)
20. return groups
```

. Go Implementation — Node and Graph

```go
// dag/node.go
package dag

import (
    "errors"
    "sync"
)

// Node represents a single nano-task in the dependency graph.
type Node struct {
    ID           string   `json:"id"`
    Dependencies []string `json:"dependencies"`

    // Mutable runtime state:
    Status       NodeStatus `json:"status"`
    OutputPath   string     `json:"output_path,omitempty"`
    OutputHash   string     `json:"output_hash,omitempty"`
    ErrorMessage string     `json:"error_message,omitempty"`
    Attempts     int        `json:"attempts"`
    StartedAt    time.Time  `json:"started_at,omitempty"`
    CompletedAt  time.Time  `json:"completed_at,omitempty"`
    WorkerID     string     `json:"worker_id,omitempty"`
}

type NodeStatus string
const (
    StatusPending    NodeStatus = "pending"
    StatusReady      NodeStatus = "ready"
    StatusRunning    NodeStatus = "running"
    StatusCompleted  NodeStatus = "completed"
    StatusFailed     NodeStatus = "failed"
    StatusSkipped    NodeStatus = "skipped"
)

// Graph is the full dependency graph for a workspace (all binders).
type Graph struct {
    mu    sync.RWMutex
    Nodes map[string]*Node       `json:"nodes"`
    Edges map[string][]string    `json:"edges"`  // id → [dependencies]
    RevEdges map[string][]string `json:"-"`      // id → [dependents]
}

// NewGraph creates an empty DAG.
func NewGraph() *Graph {
    return &Graph{
```

```
        Nodes:    make(map[string]*Node),
        Edges:    make(map[string][]string),
        RevEdges: make(map[string][]string),
    }
}

// AddNode inserts a nano-task into the graph.
// Returns an error if a cycle would be created.
func (g *Graph) AddNode(n *Node) error {
    g.mu.Lock()
    defer g.mu.Unlock()

    if _, exists := g.Nodes[n.ID]; exists {
        return fmt.Errorf("node %q already exists", n.ID)
    }

    g.Nodes[n.ID] = n
    g.Edges[n.ID] = n.Dependencies

    for _, dep := range n.Dependencies {
        g.RevEdges[dep] = append(g.RevEdges[dep], n.ID)
    }

    if cycle := g.detectCycle(n.ID); cycle != nil {
        // Roll back
        delete(g.Nodes, n.ID)
        delete(g.Edges, n.ID)
        for _, dep := range n.Dependencies {
            g.removeRevEdge(dep, n.ID)
        }
        return fmt.Errorf("adding node %q would create a cycle: %v", n.ID, cycle)
    }
    return nil
}

// TopologicalSort returns all nodes in dependency order (Kahn's algorithm).
func (g *Graph) TopologicalSort() ([]string, error) {
    g.mu.RLock()
    defer g.mu.RUnlock()

    inDegree := make(map[string]int, len(g.Nodes))
    for id, deps := range g.Edges {
        inDegree[id] = len(deps)
    }

    queue := make([]string, 0)
```

```
    for id := range g.Nodes {
        if inDegree[id] == 0 {
            queue = append(queue, id)
        }
    }

    sorted := make([]string, 0, len(g.Nodes))
    for len(queue) > 0 {
        current := queue[0]
        queue = queue[1:]
        sorted = append(sorted, current)

        for _, dependent := range g.RevEdges[current] {
            inDegree[dependent]--
            if inDegree[dependent] == 0 {
                queue = append(queue, dependent)
            }
        }
    }

    if len(sorted) != len(g.Nodes) {
        return nil, errors.New("cycle detected in dependency graph")
    }
    return sorted, nil
}

// ParallelGroups partitions sorted nodes into parallel execution groups.
func (g *Graph) ParallelGroups() ([][]string, error) {
    sorted, err := g.TopologicalSort()
    if err != nil {
        return nil, err
    }

    g.mu.RLock()
    defer g.mu.RUnlock()

    depth := make(map[string]int, len(sorted))
    for _, id := range sorted {
        deps := g.Edges[id]
        if len(deps) == 0 {
            depth[id] = 0
            continue
        }
        maxDep := 0
        for _, dep := range deps {
            if d, ok := depth[dep]; ok && d > maxDep {
```

```
                maxDep = d
            }
        }
        depth[id] = maxDep + 1
    }

    groups := make([][]string, 0)
    var currentDepth int
    var currentGroup []string
    for i, id := range sorted {
        if i == 0 || depth[id] != currentDepth {
            if len(currentGroup) > 0 {
                groups = append(groups, currentGroup)
            }
            currentGroup = []string{id}
            currentDepth = depth[id]
        } else {
            currentGroup = append(currentGroup, id)
        }
    }
    if len(currentGroup) > 0 {
        groups = append(groups, currentGroup)
    }
    return groups, nil
}

// detectCycle checks whether adding a path through 'origin' creates a cycle.
func (g *Graph) detectCycle(origin string) []string {
    visited := make(map[string]bool)
    path := make([]string, 0)

    var dfs func(id string) bool
    dfs = func(id string) bool {
        if id == origin {
            return true
        }
        if visited[id] {
            return false
        }
        visited[id] = true
        path = append(path, id)
        for _, dep := range g.Edges[id] {
            if dfs(dep) {
                return true
            }
        }
```

```
        path = path[:len(path)-1]
        return false
    }
    if dfs(origin) {
        return path
    }
    return nil
}

func (g *Graph) removeRevEdge(dep, target string) {
    deps := g.RevEdges[dep]
    for i, d := range deps {
        if d == target {
            g.RevEdges[dep] = append(deps[:i], deps[i+1:]...)
            return
        }
    }
}

// ReadyNodes returns all nodes whose dependencies are all completed.
func (g *Graph) ReadyNodes() []*Node {
    g.mu.RLock()
    defer g.mu.RUnlock()

    ready := make([]*Node, 0)
    for _, node := range g.Nodes {
        if node.Status != StatusPending {
            continue
        }
        allDepsComplete := true
        for _, dep := range node.Dependencies {
            depNode, ok := g.Nodes[dep]
            if !ok || depNode.Status != StatusCompleted {
                allDepsComplete = false
                break
            }
        }
        if allDepsComplete {
            nodeCopy := *node
            nodeCopy.Status = StatusReady
            ready = append(ready, &nodeCopy)
        }
    }
    return ready
}
```

. TypeScript Implementation — Lightweight In-Browser DAG

```ts
// dag/graph.ts
export type NodeStatus =
  | "pending"
  | "ready"
  | "running"
  | "completed"
  | "failed"
  | "skipped";

export interface NanoTaskNode {
  id: string;
  dependencies: string[];
  status: NodeStatus;
  outputPath?: string;
  outputHash?: string;
  errorMessage?: string;
  attempts: number;
  startedAt?: Date;
  completedAt?: Date;
}

export class TaskGraph {
  private nodes: Map<string, NanoTaskNode> = new Map();
  private edges: Map<string, string[]> = new Map();
  private revEdges: Map<string, string[]> = new Map();

  addNode(node: NanoTaskNode): void {
    if (this.nodes.has(node.id)) {
      throw new Error(`Node "${node.id}" already exists`);
    }
    this.nodes.set(node.id, { ...node });

    const deps = node.dependencies;
    this.edges.set(node.id, deps);
    for (const dep of deps) {
      const existing = this.revEdges.get(dep) ?? [];
      existing.push(node.id);
      this.revEdges.set(dep, existing);
    }

    if (this.detectCycle(node.id)) {
      this.nodes.delete(node.id);
      this.edges.delete(node.id);
      for (const dep of deps) {
        this.removeRevEdge(dep, node.id);
```

```
    }
    throw new Error(`Adding "${node.id}" would create a cycle`);
  }
 }

 topologicalSort(): string[] {
  const inDegree = new Map<string, number>();
  for (const [id, deps] of this.edges) {
   inDegree.set(id, deps.length);
  }

  const queue: string[] = [];
  for (const id of this.nodes.keys()) {
   if ((inDegree.get(id) ?? 0) === 0) {
    queue.push(id);
   }
  }

  const sorted: string[] = [];
  while (queue.length > 0) {
   const current = queue.shift()!;
   sorted.push(current);

   for (const dependent of this.revEdges.get(current) ?? []) {
    const deg = (inDegree.get(dependent) ?? 0) - 1;
    inDegree.set(dependent, deg);
    if (deg === 0) {
     queue.push(dependent);
    }
   }
  }

  if (sorted.length !== this.nodes.size) {
   throw new Error("Cycle detected in dependency graph");
  }
  return sorted;
 }

 parallelGroups(): string[][] {
  const sorted = this.topologicalSort();
  const depth = new Map<string, number>();

  for (const id of sorted) {
   const deps = this.edges.get(id) ?? [];
   if (deps.length === 0) {
    depth.set(id, 0);
```

```
    } else {
      const maxDep = Math.max(...deps.map(d => depth.get(d) ?? 0));
      depth.set(id, maxDep + 1);
    }
  }

  const groups: string[][] = [];
  let currentDepth = -1;
  let currentGroup: string[] = [];

  for (const id of sorted) {
    const d = depth.get(id) ?? 0;
    if (d !== currentDepth) {
      if (currentGroup.length > 0) groups.push(currentGroup);
      currentGroup = [id];
      currentDepth = d;
    } else {
      currentGroup.push(id);
    }
  }
  if (currentGroup.length > 0) groups.push(currentGroup);
  return groups;
}

getReadyNodes(): NanoTaskNode[] {
  const ready: NanoTaskNode[] = [];
  for (const node of this.nodes.values()) {
    if (node.status !== "pending") continue;
    const allDone = node.dependencies.every(dep => {
      const depNode = this.nodes.get(dep);
      return depNode?.status === "completed";
    });
    if (allDone) {
      ready.push({ ...node, status: "ready" });
    }
  }
  return ready;
}

private detectCycle(origin: string): boolean {
  const visited = new Set<string>();
  const dfs = (id: string): boolean => {
    if (id === origin) return true;
    if (visited.has(id)) return false;
    visited.add(id);
    for (const dep of this.edges.get(id) ?? []) {
```

```
        if (dfs(dep)) return true;
      }
      return false;
    };
    return dfs(origin);
  }

  private removeRevEdge(dep: string, target: string): void {
    const deps = this.revEdges.get(dep) ?? [];
    this.revEdges.set(dep, deps.filter(d => d !== target));
  }
}
```

5

---

5 1

. Decomposition Algorithm

. Overview

The decomposer takes a SpecKit `tasks.md` file (or a single task definition) and produces the set of YAML nano-task files. The decomposition is **multi-pass**:

1. **Pass 1 — Parse.** Extract tasks from Markdown task lists.
2. **Pass 2 — Coarse decomposition.** Break each task into coarse sub-tasks (4–8 per task).
3. **Pass 3 — Fine decomposition.** Break each coarse sub-task into nano-tasks, each ≤512 tokens.
4. **Pass 4 — Cross-cutting extraction.** Identify cross-cutting concerns (auth, logging, error handling, validation) and extract them as shared nano-tasks.
5. **Pass 5 — Binding generation.** Generate binder-level integration tasks and cross-binder dependency edges.
6. **Pass 6 — Validation.** Topological sort, cycle check, token budget verification, schema validation.

```
.    Pass      — Parse SpecKit Tasks
```

```go
// decompose/parser.go
package decompose

import (
    "bufio"
    "os"
    "regexp"
    "strings"
)

// Task represents a single SpecKit task extracted from tasks.md.
type Task struct {
    ID          string   // e.g. "T001" or "auth-impl"
    Title       string
    Description string   // Full markdown description
    DependsOn   []string // Other task IDs this task depends on
    Subtasks    []string // Raw subtask lines from the markdown
    Priority    string   // "P0" / "P1" / "P2"
    EstimatedHours float64
}

// ParseTasksMD reads a SpecKit tasks.md file and extracts structured tasks.
func ParseTasksMD(path string) ([]Task, error) {
    f, err := os.Open(path)
    if err != nil {
        return nil, err
    }
    defer f.Close()

    var tasks []Task
    var current *Task
    hdrRe := regexp.MustCompile(`^##\s+(.+)$`)
    subRe := regexp.MustCompile(`^-\s+\[ \]\s+(.+)$`)     // unchecked
    depRe := regexp.MustCompile(`depends on:\s*(.+)$`)

    scanner := bufio.NewScanner(f)
    for scanner.Scan() {
        line := scanner.Text()

        if matches := hdrRe.FindStringSubmatch(line); matches != nil {
            if current != nil {
                tasks = append(tasks, *current)
            }
            current = &Task{Title: matches[1]}
            // Extract ID from title: "T001: User auth" → "T001"
```

```
            if parts := strings.SplitN(matches[1], ":", 2); len(parts) > 1 {
                current.ID = strings.TrimSpace(parts[0])
                current.Title = strings.TrimSpace(parts[1])
            }
            continue
        }

        if current == nil {
            continue
        }

        if matches := subRe.FindStringSubmatch(line); matches != nil {
            current.Subtasks = append(current.Subtasks, strings.TrimSpace(matches[1]))
            continue
        }

        if matches := depRe.FindStringSubmatch(line); matches != nil {
            for _, dep := range strings.Split(matches[1], ",") {
                current.DependsOn = append(current.DependsOn, strings.TrimSpace(dep))
            }
            continue
        }

        current.Description += line + "\n"
    }
    if current != nil {
        tasks = append(tasks, *current)
    }
    return tasks, scanner.Err()
}
```

. Pass — Coarse Decomposition

Each SpecKit task is decomposed into 4–8 coarse sub-tasks by the LLM. The decomposer sends the task description to the LLM with a system prompt instructing it to produce a JSON array of coarse sub-tasks.

```ts
// decompose/coarse.ts
export interface CoarseSubTask {
  id: string;
  title: string;
  description: string;
  estimatedNanoTasks: number;
  tokenBudget: number;
}

export async function decomposeCoarse(task: SpecKitTask): Promise<CoarseSubTask[]> {
  const prompt = `
You are a task decomposition engine.  Given a SpecKit task, break it into
4–8 coarse sub-tasks.  Each coarse sub-task will later be decomposed into
nano-tasks of ≤512 tokens each.

TASK:
Title: ${task.title}
Description: ${task.description}
Subtasks: ${task.subtasks.join("\n")}

Output a JSON array of objects with these fields:
- id: kebab-case id (parented under the task id, e.g. "T001-validate")
- title: one sentence
- description: 3–5 sentences
- estimatedNanoTasks: how many nano-tasks this will yield (1–5)
- tokenBudget: estimated total tokens (sum of all future nano-tasks)

Respond ONLY with the JSON array.  No markdown, no explanation.
`.trim();

  const response = await llm.complete(prompt, { maxTokens: 2000 });
  return JSON.parse(response) as CoarseSubTask[];
}
```

. Pass — Nano Decomposition

Each coarse sub-task is decomposed into 1–5 nano-tasks by the LLM. The system prompt enforces:

- ≤512 tokens per nano-task.
- Every nano-task has an explicit input/output JSON Schema.
- Every nano-task has a RED-first test template.

- Every nano-task declares its dependencies explicitly.

```ts
// decompose/nano.ts
export interface NanoTaskSpec {
  id: string;
  title: string;
  description: string;
  type: "function" | "integration" | "stateful" | "binding" | "test-only";
  inputSchema: object;
  inputExample: object;
  outputSchema: object;
  outputExample: object;
  dependencies: string[];
  estimatedTokens: number;
  testTemplate: string;
  implementationHints: string[];
  acceptance: string[];
}

export async function decomposeNano(
  coarse: CoarseSubTask,
  parentBinderId: string,
  existingNanoIds: Set<string>
): Promise<NanoTaskSpec[]> {
  const prompt = `
DECOMPOSE the following coarse sub-task into 1–5 fully self-contained
nano-tasks.  Each nano-task MUST:

1. Be ≤512 tokens in its implementation surface (the code the LLM writes).
2. Have a typed input/output contract (JSON Schema subset).
3. Have a RED-first test template that MUST fail before implementation.
4. Declare its dependencies by nano-task id (which must already exist
   or be earlier in this decomposition batch).
5. Be implementable with ZERO knowledge of any other nano-task's internals
   — only read upstream outputs by their public schema.

COARSE SUB-TASK:
Id: ${coarse.id}
Title: ${coarse.title}
Description: ${coarse.description}

EXISTING nano-task ids (you may depend on these):
${[...existingNanoIds].join(", ") || "(none yet)"}

PARENT binder id: ${parentBinderId}

RESPOND with a JSON array of nano-task specs in this exact schema:
```

```
[
  {
    "id": "kebab-case-id",
    "title": "one sentence",
    "description": "4-8 sentence plain-language description",
    "type": "function|integration|stateful|binding|test-only",
    "inputSchema": { "type": "object", "properties": {...}, "required": [...] },
    "inputExample": { ... },
    "outputSchema": { "type": "object", "properties": {...} },
    "outputExample": { ... },
    "dependencies": ["nano-xyz", ...],
    "estimatedTokens": 384,
    "testTemplate": "// RED test that must fail before impl exists\\n...",
    "implementationHints": ["hint 1", "hint 2"],
    "acceptance": ["criterion 1", "criterion 2", "criterion 3"]
  }
]

Respond ONLY with the JSON array.  No markdown, no explanation.
`.trim();

  const response = await llm.complete(prompt, { maxTokens: 4000 });
  const specs = JSON.parse(response) as NanoTaskSpec[];

  // Validate
  for (const spec of specs) {
    if (spec.estimatedTokens > 512) {
      throw new Error(`nano-task ${spec.id} exceeds token budget: ${spec.estimatedTokens}`);
    }
    for (const dep of spec.dependencies) {
      if (!existingNanoIds.has(dep) && !specs.some(s => s.id === dep)) {
        throw new Error(`nano-task ${spec.id} depends on unknown ${dep}`);
      }
    }
    existingNanoIds.add(spec.id);
  }
  return specs;
}
```

. Token Budget Enforcement

The 512-token ceiling per nano-task is the single most important constraint. It is derived from:

- **Local LLM context window:** 4096–8192 tokens (7B–13B quantized models).
- **System prompt overhead:** ~1500 tokens (governance preamble, skill instructions).
- **Test template:** ~200 tokens.
- **Implementation surface:** the remaining budget.

The budget is enforced at two points:

1. **Decomposition time** — the LLM estimates `estimated_tokens`. The decomposer refuses nano-tasks exceeding 512, re-decomposing them.
2. **Post-implementation verification** — the executor counts the actual implementation tokens and FLAGs violations for re-decomposition.

```
Token Budget = 512 tokens
┌──────────────────────────┬──────────┬──────────────────┐
│ System Prompt            │ Test     │ Implementation   │
│ ~1500 tokens             │ ~200 tok │ ≤512 tokens      │
│ (outside per-task budget)│          │                  │
└──────────────────────────┴──────────┴──────────────────┘
       Total context ≤ 4096 (safe for 7B models)
```

. Cross-Cutting Concern Extraction

During Pass 4, the decomposer identifies concerns that appear across multiple nano-tasks and extracts them:

- **Validation functions** (password strength, email format, URL safety).
- **Error types** (typed error classes/structs used by multiple nano-tasks).
- **Logging/tracing** (common log format or tracing span helpers).
- **Serialization** (shared JSON/protobuf marshal/unmarshal).
- **Configuration** (shared config struct, env-var reader).

These become their own nano-tasks, and the nano-tasks that used to inline them now declare dependencies on the extracted nano-tasks.

```go
// decompose/crosscut.go
type CrossCuttingConcern struct {
    Name        string
    Description string
    AffectedIDs []string      // nano-task ids that use this concern
    ExtractedID string        // new nano-task id for the concern
}

func ExtractCrossCutting(nanoTasks []NanoTask) []CrossCuttingConcern {
    concerns := []CrossCuttingConcern{}

    // Pattern 1: repeated "validate X" patterns
    validatePatterns := findRepeatedPatterns(nanoTasks, `validate\s+\w+`)
    for _, p := range validatePatterns {
        concerns = append(concerns, CrossCuttingConcern{
            Name:        p.name,
            Description: fmt.Sprintf("Extracted validation: %s", p.name),
            AffectedIDs: p.ids,
            ExtractedID: fmt.Sprintf("xcut-validate-%s", slugify(p.name)),
        })
    }

    // Pattern 2: repeated error type definitions
    errorTypes := findRepeatedPatterns(nanoTasks, `type\s+\w+Error\s+struct`)
    for _, e := range errorTypes {
        concerns = append(concerns, CrossCuttingConcern{
            Name:        e.name,
            Description: fmt.Sprintf("Extracted error type: %s", e.name),
            AffectedIDs: e.ids,
            ExtractedID: fmt.Sprintf("xcut-error-%s", slugify(e.name)),
        })
    }

    // Pattern 3: identical config reads
    configPatterns := findRepeatedPatterns(nanoTasks, `os\.Getenv|config\.Get|viper\.Get`)
    if len(configPatterns) >= 3 {
        concerns = append(concerns, CrossCuttingConcern{
            Name:        "config",
            Description: "Extracted configuration loading",
            AffectedIDs: flatten(configPatterns),
            ExtractedID: "xcut-config-loader",
        })
    }
```

```
    return concerns
}
```

. Full Decomposition Pipeline

```go
// decompose/pipeline.go
func DecomposePipeline(tasksMD string, workspaceDir string) (*Workspace, error) {
    // Pass 1: Parse
    tasks, err := ParseTasksMD(tasksMD)
    if err != nil {
        return nil, fmt.Errorf("pass 1 parse: %w", err)
    }

    workspace := NewWorkspace(workspaceDir)

    for _, task := range tasks {
        // Pass 2: Coarse decomposition
        coarse, err := decomposeCoarse(task)
        if err != nil {
            return nil, fmt.Errorf("pass 2 coarse %s: %w", task.ID, err)
        }

        binder := NewBinder(task)
        existingIDs := make(map[string]bool)

        for _, c := range coarse {
            // Pass 3: Nano decomposition
            nanos, err := decomposeNano(c, binder.ID, existingIDs)
            if err != nil {
                return nil, fmt.Errorf("pass 3 nano %s: %w", c.ID, err)
            }

            for _, nano := range nanos {
                binder.AddNanoTask(nano)
                workspace.Graph.AddNode(nanoToNode(nano))
                existingIDs[nano.ID] = true
            }
        }

        workspace.AddBinder(binder)
    }

    // Pass 4: Cross-cutting extraction
    allNanos := workspace.AllNanoTasks()
    concerns := ExtractCrossCutting(allNanos)
    for _, cc := range concerns {
        workspace.ExtractConcern(cc)
    }

    // Pass 5: Binding generation
```

```
    for _, binder := range workspace.Binders {
        binder.GenerateBindingNanoTask()
    }

    // Pass 6: Validate
    if _, err := workspace.Graph.TopologicalSort(); err != nil {
        return nil, fmt.Errorf("pass 6 validate: cycle detected: %w", err)
    }
    for _, nano := range workspace.AllNanoTasks() {
        if nano.EstimatedTokens > 512 {
            return nil, fmt.Errorf("pass 6 validate: %s exceeds token budget (%d > 512)",
                nano.ID, nano.EstimatedTokens)
        }
    }

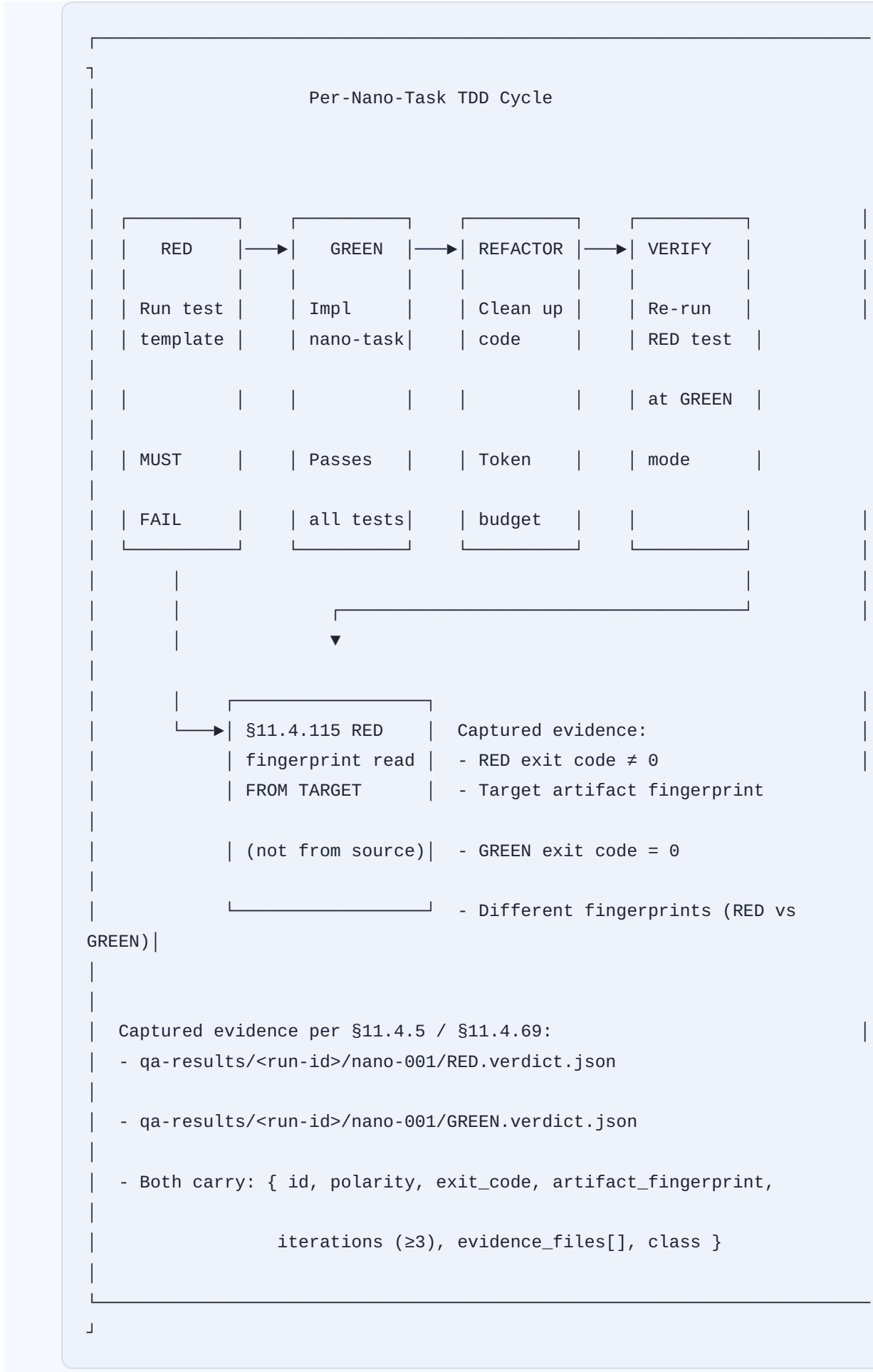
    return workspace, nil
}
```

---

6 1

. Execution Engine

. TDD Cycle Per Nano-Task


Per-Nano-Task TDD Cycle
RED
GREEN
REFACTOR
VERIFY
Run test template
Impl nano-task
Clean up code
Re-run RED test at GREEN mode
MUST FAIL
Passes all tests
Token budget
§11.4.115 RED fingerprint read FROM TARGET (not from source)
Captured evidence:
- RED exit code ≠ 0
- Target artifact fingerprint
- GREEN exit code = 0
- Different fingerprints (RED vs GREEN)
Captured evidence per §11.4.5 / §11.4.69:
- qa-results/<run-id>/nano-001/RED.verdict.json
- qa-results/<run-id>/nano-001/GREEN.verdict.json
- Both carry: { id, polarity, exit_code, artifact_fingerprint,
iterations (≥3), evidence_files[], class }

. Executor — Main Loop

```go
// executor/executor.go
package executor

import (
    "context"
    "fmt"
    "log"
    "sync"
    "time"
)

type Executor struct {
    Graph       *dag.Graph
    OutputDir   string
    MaxParallel int
    Logger      *log.Logger

    mu      sync.Mutex
    running int
    errors  []error
}

func NewExecutor(g *dag.Graph, outputDir string, maxParallel int) *Executor {
    return &Executor{
        Graph:       g,
        OutputDir:   outputDir,
        MaxParallel: maxParallel,
        Logger:      log.Default(),
    }
}

// Run executes all nano-tasks in the graph using TDD cycles.
// Returns the first error that prevents forward progress.
func (e *Executor) Run(ctx context.Context) error {
    for {
        ready := e.Graph.ReadyNodes()
        if len(ready) == 0 {
            break
        }

        // Group by topological level for parallelism
        groups := e.groupByDepth(ready)
        for _, group := range groups {
            if err := e.executeGroup(ctx, group); err != nil {
                return err
```

```
            }
        }
    }

    // Check completion
    incomplete := e.countIncomplete()
    if incomplete > 0 {
        return fmt.Errorf("%d nano-tasks did not reach completed status", incomplete)
    }
    return nil
}

func (e *Executor) executeGroup(ctx context.Context, nodes []*dag.Node) error {
    sem := make(chan struct{}, e.MaxParallel)
    var wg sync.WaitGroup
    errCh := make(chan error, len(nodes))

    for _, node := range nodes {
        sem <- struct{}{} // acquire
        wg.Add(1)
        go func(n *dag.Node) {
            defer wg.Done()
            defer func() { <-sem }() // release

            if err := e.executeNanoTask(ctx, n); err != nil {
                errCh <- err
            }
        }(node)
    }
    wg.Wait()
    close(errCh)

    // Collect first error (others are likely cascading)
    for err := range errCh {
        return err
    }
    return nil
}

func (e *Executor) executeNanoTask(ctx context.Context, node *dag.Node) error {
    // Mark running
    e.markStatus(node.ID, dag.StatusRunning)

    // Collect dependency outputs
    depOutputs := make(map[string]interface{})
    for _, depID := range node.Dependencies {
```

```
        output, err := e.loadDependencyOutput(depID)
        if err != nil {
            return fmt.Errorf("nano %s: failed to load dep %s: %w", node.ID, depID, err)
        }
        depOutputs[depID] = output
    }

    // ── RED phase ──
    redStart := time.Now()
    redResult, err := e.runTest(ctx, node, depOutputs, "RED")
    if err != nil {
        return fmt.Errorf("nano %s: RED test infrastructure error: %w", node.ID, err)
    }
    if redResult.ExitCode == 0 {
        // RED test passed — the test is broken (should fail before impl)
        return fmt.Errorf("nano %s: RED test PASSED — test is not catching "+
            "the absent implementation (bluff gate, §11.4.115(F))", node.ID)
    }
    e.Logger.Printf("[%s] RED phase: FAIL (expected) in %v", node.ID, time.Since(redStart))

    // ── IMPLEMENT phase ──
    implStart := time.Now()
    implResult, err := e.dispatchImplementation(ctx, node, depOutputs)
    if err != nil {
        return fmt.Errorf("nano %s: implementation phase failed: %w", node.ID, err)
    }
    e.Logger.Printf("[%s] IMPL phase: done in %v, %d tokens",
        node.ID, time.Since(implStart), implResult.TokensUsed)

    // Verify token budget
    if implResult.TokensUsed > 512 {
        e.Logger.Printf("[%s] WARNING: implementation used %d tokens "+
            "(budget: 512) — flagging for re-decomposition", node.ID, implResult.TokensUsed)
        // Flag, don't fail — the nano-task MAY still work
    }

    // ── GREEN phase ──
    greenStart := time.Now()
    greenResult, err := e.runTest(ctx, node, depOutputs, "GREEN")
    if err != nil {
        return fmt.Errorf("nano %s: GREEN test infrastructure error: %w", node.ID, err)
    }
    if greenResult.ExitCode != 0 {
        return fmt.Errorf("nano %s: GREEN test FAILED — implementation is incorrect: %s",
            node.ID, greenResult.Stderr)
    }
```

```
    e.Logger.Printf("[%s] GREEN phase: PASS in %v", node.ID, time.Since(greenStart))

    // ── VERIFY phase (re-run 3× for determinism §11.4.50) ──
    for i := 0; i < 3; i++ {
        result, err := e.runTest(ctx, node, depOutputs, "VERIFY")
        if err != nil || result.ExitCode != 0 {
            return fmt.Errorf("nano %s: VERIFY iteration %d/%d FAILED — "+
                "non-deterministic behavior (§11.4.50 violation)", node.ID, i+1, 3)
        }
    }

    // Mark completed
    e.markStatus(node.ID, dag.StatusCompleted)
    return nil
}
```

- Error Handling and Retry

```go
func (e *Executor) executeWithRetry(ctx context.Context, node *dag.Node) error {
    policy := node.RetryPolicy
    for attempt := 1; attempt <= policy.MaxAttempts; attempt++ {
        err := e.executeNanoTask(ctx, node)
        if err == nil {
            return nil
        }

        // Classify the error
        if isTransient(err) {
            // Rate-limit, network blip, OOM-killed worker — retry with backoff
            backoff := e.computeBackoff(policy, attempt)
            e.Logger.Printf("[%s] transient error (attempt %d/%d): %v — retrying in %v",
                node.ID, attempt, policy.MaxAttempts, err, backoff)
            select {
            case <-time.After(backoff):
                continue
            case <-ctx.Done():
                return ctx.Err()
            }
        }

        // Non-transient: compile error, type mismatch, logic error
        return fmt.Errorf("nano %s: terminal error on attempt %d: %w", node.ID, attempt, err)
    }
    return fmt.Errorf("nano %s: exhausted %d attempts", node.ID, policy.MaxAttempts)
}

func isTransient(err error) bool {
    // Rate-limit, network, OOM, worker crash
    s := err.Error()
    return strings.Contains(s, "rate limit") ||
        strings.Contains(s, "connection reset") ||
        strings.Contains(s, "OOM") ||
        strings.Contains(s, "signal: killed")
}

func (e *Executor) computeBackoff(policy RetryPolicy, attempt int) time.Duration {
    switch policy.Backoff {
    case "exponential":
        return time.Duration(policy.BackoffBaseMs*(1<<uint(attempt-1))) * time.Millisecond
    case "linear":
        return time.Duration(policy.BackoffBaseMs*attempt) * time.Millisecond
    default: // "fixed"
        return time.Duration(policy.BackoffBaseMs) * time.Millisecond
```

```
    }
}
```

- Progress Tracking and Resumption

```go
// executor/checkpoint.go
type Checkpoint struct {
    WorkspaceID    string            `json:"workspace_id"`
    GraphSnapshot  map[string]string  `json:"graph_snapshot"` // node ID → status
    CompletedAt    time.Time          `json:"completed_at"`
    EvidenceDir    string             `json:"evidence_dir"`
}

// SaveCheckpoint persists the current execution state so a crashed
// executor can resume without re-running completed nano-tasks.
func (e *Executor) SaveCheckpoint() error {
    checkpoint := Checkpoint{
        WorkspaceID:   e.Graph.WorkspaceID,
        GraphSnapshot: make(map[string]string),
        CompletedAt:   time.Now().UTC(),
        EvidenceDir:   filepath.Join(e.OutputDir, "checkpoints"),
    }
    for id, node := range e.Graph.Nodes {
        checkpoint.GraphSnapshot[id] = string(node.Status)
    }

    path := filepath.Join(e.OutputDir, "checkpoints", "latest.json")
    data, err := json.MarshalIndent(checkpoint, "", "  ")
    if err != nil {
        return err
    }
    return atomicWriteFile(path, data, 0644)
}

// ResumeFromCheckpoint restores execution state from the last checkpoint.
func (e *Executor) ResumeFromCheckpoint() error {
    path := filepath.Join(e.OutputDir, "checkpoints", "latest.json")
    data, err := os.ReadFile(path)
    if err != nil {
        return err
    }
    var cp Checkpoint
    if err := json.Unmarshal(data, &cp); err != nil {
        return err
    }
    for id, status := range cp.GraphSnapshot {
        if node, ok := e.Graph.Nodes[id]; ok {
            node.Status = dag.NodeStatus(status)
        }
    }
```

```
    e.Logger.Printf("resumed: %d nodes restored from checkpoint", len(cp.GraphSnapshot))
    return nil
}

// atomicWriteFile writes data to a temp file then renames to avoid torn writes.
func atomicWriteFile(path string, data []byte, perm os.FileMode) error {
    tmp := path + ".tmp"
    if err := os.WriteFile(tmp, data, perm); err != nil {
        return err
    }
    return os.Rename(tmp, path)
}
```

. Integration with SpecKit

. The `speckit.helix-nano-bridge.decompose` Command

This command is the entry point that takes a SpecKit workspace and produces the nano-task workspace.

```
Usage:
  speckit.helix-nano-bridge.decompose [--workspace <path>] [--output
<path>]
                                      [--model <model>] [--max-tokens 512]

Options:
  --workspace     Path to the SpecKit workspace (contains tasks.md).
Default: cwd.
  --output        Path to write nano-task workspace.  Default: .nano-
tasks/
  --model         LLM model to use for decomposition.  Default: configured
default.
  --max-tokens    Maximum tokens per nano-task.  Default: 512.
  --dry-run       Parse and validate without dispatching to LLM.

Workflow:
  1. Read tasks.md from the SpecKit workspace.
  2. For each task, decompose into coarse → nano → cross-cutting.
  3. Generate binder tasks with integration tests.
  4. Build the full DAG and validate (no cycles, all deps resolvable).
  5. Write nano-task YAML files to the output directory.
  6. Generate execution plan (ordered groups for the executor).

Output structure:
  .nano-tasks/
  ├── workspace.yaml          # Workspace manifest
  ├── binders/
  │   ├── binder-auth.yaml    # One binder per SpecKit task
  │   └── binder-session.yaml
  ├── nanos/
  │   ├── nano-000.yaml       # One YAML file per nano-task
  │   ├── nano-001.yaml
  │   └── ...
  ├── graph.json              # Serialized DAG for the executor
  └── execution-plan.json     # Topologically sorted groups
```

. CLI Implementation

```go
// cmd/decompose/main.go
package main

import (
    "flag"
    "fmt"
    "os"

    "github.com/HelixDevelopment/helix_nano_bridge/decompose"
)

func main() {
    workspace := flag.String("workspace", ".", "SpecKit workspace path")
    output := flag.String("output", ".nano-tasks", "Output directory")
    maxTokens := flag.Int("max-tokens", 512, "Token budget per nano-task")
    dryRun := flag.Bool("dry-run", false, "Parse and validate only")
    flag.Parse()

    if *dryRun {
        tasks, err := decompose.ParseTasksMD(*workspace + "/tasks.md")
        if err != nil {
            fmt.Fprintf(os.Stderr, "parse error: %v\n", err)
            os.Exit(1)
        }
        fmt.Printf("Parsed %d tasks from tasks.md\n", len(tasks))
        for _, t := range tasks {
            fmt.Printf("  %s: %s (%d subtasks)\n", t.ID, t.Title, len(t.Subtasks))
        }
        return
    }

    ws, err := decompose.DecomposePipeline(
        *workspace+"/tasks.md",
        *output,
    )
    if err != nil {
        fmt.Fprintf(os.Stderr, "decomposition error: %v\n", err)
        os.Exit(1)
    }

    fmt.Printf("Decomposition complete:\n")
    fmt.Printf("  Binders:    %d\n", len(ws.Binders))
    fmt.Printf("  Nano-tasks: %d\n", len(ws.AllNanoTasks()))
    fmt.Printf("  DAG depth:  %d\n", ws.Graph.MaxDepth())
    fmt.Printf("  Cycles:     none\n")
```

```
    fmt.Printf("  Output:    %s\n", *output)
}
```

8.1. Integration with Superpowers

8.1.1. Skill Mapping

The Superpowers skills map onto nano-task execution as follows:

Superpowers Skill	Nano-Task Phase	Description
`test-driven-development`	RED + GREEN + VERIFY	Drives the per-nano-task TDD cycle
`systematic-debugging`	On FAIL	Activates when any nano-task GREEN fails
`subagent-driven-development`	Parallel groups	Each subagent claims nano-tasks from a group
`writing-plans`	Decomposition	Author the decomposition plan
`receiving-code-review`	Post-GREEN	§11.4.142 independent review of implementation
`finishing-a-development-branch`	Binder completion	When all nano-tasks in a binder are GREEN

- Subagent Dispatch

```
// superpowers/dispatch.go
func DispatchNanoTaskToSubagent(
    ctx context.Context,
    node *dag.Node,
    superpowers *SuperpowersClient,
) (*SubagentResult, error) {
    // Build the subagent prompt from the nano-task YAML
    taskYAML, err := yaml.Marshal(node)
    if err != nil {
        return nil, err
    }

    prompt := fmt.Sprintf(`
You are a Subagent executing a SINGLE nano-task in the Helix Nano-Task Engine.

YOUR NANO-TASK:
%s

DEPENDENCY OUTPUTS (already completed, read-only):
%s

INSTRUCTIONS:
1. FIRST, run the test template in RED mode.  It MUST fail.
2. Implement the nano-task to satisfy the test.
3. Run the test in GREEN mode.  It MUST pass (exit 0).
4. Re-run 3 times for determinism proof (§11.4.50).
5. Write the implementation to the declared file_scope paths.
6. Write verdict files to qa-results/<run-id>/<nano-id>/{RED,GREEN}.verdict.json

RESPOND ONLY with the implementation code and verdict paths.
`, taskYAML, formatDependencyOutputs(node))

    // Dispatch to the appropriate model based on task type
    model := selectModel(node)

    result, err := superpowers.Dispatch(ctx, &DispatchRequest{
        Skill:      "test-driven-development",
        Prompt:     prompt,
        Model:      model,
        Effort:     "xhigh",
        Timeout:    time.Duration(node.TimeoutMS) * time.Millisecond,
        OutputFile: node.OutputPath,
    })
    if err != nil {
        return nil, err
```

```
    }

    return &SubagentResult{
        NodeID:     node.ID,
        ExitCode:   result.ExitCode,
        TokensUsed: result.TokensUsed,
        OutputPath: result.OutputPath,
        Verdicts:   result.Verdicts,
    }, nil
}
```

. Systematic Debugging Integration

When a nano-task's GREEN test fails, the `systematic-debugging` skill activates per §11.4.102(D):

```
func (e *Executor) handleGreenFailure(ctx context.Context, node *dag.Node, err error) error {
    e.Logger.Printf("[%s] GREEN test FAILED — activating systematic-debugging (§11.4.102)",
                node.ID)

    debugPrompt := fmt.Sprintf(`
SYSTEMATIC DEBUGGING for nano-task %s:

FAILURE: %v

NANO-TASK SPEC:
%s

DEPENDENCY OUTPUTS:
%s

RED TEST RESULT:
%s

PHASE 1 — ROOT CAUSE:
1. Read the real failure output.
2. Reproduce the failure deterministically.
3. Gather facts: what changed between RED and GREEN?

PHASE 2 — PATTERN:
1. Classify the defect (type error / logic error / boundary / race).
2. Search the same pattern in sibling nano-tasks.

PHASE 3 — HYPOTHESIS:
1. Formulate a falsifiable root cause.
2. Prove or disprove with captured evidence.

PHASE 4 — IMPLEMENTATION:
1. Fix ONLY against the proven root cause.
2. Re-run GREEN test.

§11.4.6: NO GUESSING.  Every conclusion cites captured evidence.
`, node.ID, err, formatNodeSpec(node), formatDependencyOutputs(node),
        formatRedResult(node))

    result, err := e.Superpowers.Dispatch(ctx, &DispatchRequest{
        Skill:   "systematic-debugging",
        Prompt:  debugPrompt,
        Model:   "fable",
        Effort:  "xhigh",
        Timeout: 10 * time.Minute,
    })
```

```
    if err != nil {
        return err
    }

    if result.FixApplied {
        e.Logger.Printf("[%s] fix applied by systematic-debugging, re-running GREEN", node.ID)
        return e.executeNanoTask(ctx, node) // retry with fix
    }
    return fmt.Errorf("systematic-debugging for %s did not resolve the failure", node.ID)
}
```

9

---

9 1 11 4 93

. Workable Items Mapping

. N-Layer Mapping per § . .

Every nano-task maps to a workable item in the SQLite single-source-of-truth. The hierarchy is:

```
Project (1 root)
 └── SpecKit Feature (ATM-NNN)                    ← top-level item
      └── SpecKit Task (ATM-NNN-sub)              ← binder item
           └── Nano-Task (ATM-NNN-sub-sub)        ← nano-task item
                └── Subagent Run (diary row)      ← §11.4.149 test_diary
```

. Database Schema Extension

```sql
-- Extensions to the §11.4.93 workable_items schema for nano-task tracking

CREATE TABLE IF NOT EXISTS nano_tasks (
    nano_id         TEXT PRIMARY KEY,         -- e.g. "nano-001"
    parent_binder   TEXT NOT NULL,            -- FK → workable_items.atm_id
    spec_kit_task   TEXT NOT NULL,            -- Original SpecKit task ref
    status          TEXT NOT NULL DEFAULT 'Queued',
        CHECK(status IN (
            'Queued','In progress','Ready for testing','In testing',
            'Reopened','Completed (→ Fixed.md)'
        )),
    type            TEXT NOT NULL DEFAULT 'Task',
    yaml_path       TEXT NOT NULL,            -- Path to the nano-task YAML
    estimated_tokens INTEGER NOT NULL,
    actual_tokens   INTEGER,                  -- Populated post-implementation
    implementation_path TEXT,                 -- Path to the generated code
    test_path       TEXT,                     -- Path to the generated test

    FOREIGN KEY (parent_binder) REFERENCES workable_items(atm_id)
);

CREATE TABLE IF NOT EXISTS nano_task_dependencies (
    nano_id     TEXT NOT NULL,
    depends_on  TEXT NOT NULL,
    PRIMARY KEY (nano_id, depends_on),
    FOREIGN KEY (nano_id) REFERENCES nano_tasks(nano_id),
    FOREIGN KEY (depends_on) REFERENCES nano_tasks(nano_id)
);

CREATE TABLE IF NOT EXISTS nano_task_verdicts (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    nano_id         TEXT NOT NULL,
    polarity        TEXT NOT NULL CHECK(polarity IN ('RED','GREEN','VERIFY')),
    exit_code       INTEGER NOT NULL,
    artifact_fingerprint TEXT NOT NULL,       -- §11.4.115(F)
    evidence_path   TEXT NOT NULL,            -- §11.4.69
    iterations      INTEGER NOT NULL DEFAULT 3,
    timestamp       TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (nano_id) REFERENCES nano_tasks(nano_id)
);

-- Index for the status-custody sweep (§11.4.146(D3) Seam B)
CREATE INDEX idx_nano_verdicts_nano_polarity
    ON nano_task_verdicts(nano_id, polarity);
```

. Item Creation During Decomposition

```go
// workable/nano_items.go
func CreateNanoTaskWorkableItem(db *sql.DB, nano *NanoTask, binderATM string) error {
    atmID, err := assignNextATMID(db, "ATM")
    if err != nil {
        return fmt.Errorf("assign ATM id: %w", err)
    }

    _, err = db.Exec(`
        INSERT INTO nano_tasks (
            nano_id, parent_binder, spec_kit_task, status, type,
            yaml_path, estimated_tokens
        ) VALUES (?, ?, ?, 'Queued', 'Task', ?, ?)
    `, nano.ID, binderATM, nano.Parent, nano.YAMLPath, nano.EstimatedTokens)
    if err != nil {
        return fmt.Errorf("insert nano_task: %w", err)
    }

    // Insert into the main workable_items table for §11.4.148 integrity
    _, err = db.Exec(`
        INSERT INTO workable_items (
            atm_id, status, type, description, created_at
        ) VALUES (?, 'Queued', 'Task', ?, datetime('now'))
    `, atmID, nano.Description)
    if err != nil {
        return fmt.Errorf("insert workable_item: %w", err)
    }

    // Insert dependency edges
    for _, dep := range nano.Dependencies {
        _, err = db.Exec(`
            INSERT INTO nano_task_dependencies (nano_id, depends_on)
            VALUES (?, ?)
        `, nano.ID, dep)
        if err != nil {
            return fmt.Errorf("insert dependency %s → %s: %w", nano.ID, dep, err)
        }
    }

    nano.WorkableItemATM = atmID
    return nil
}
```

. Anti-Bluff Verification

. The Bluff Problem at Nano Scale

A nano-task can produce a "green" test while the feature the user cares about is still broken. The canonical failure modes:

1. **The test agrees with the implementation but both are wrong.** The test asserts `hash.startsWith("$2")`, the implementation returns `"$2b$not-a-real-hash"`. Both pass. The user's auth is broken.
2. **The nano-task passes in isolation but fails when composed.** Nano-001 returns a bcrypt hash. Nano-003 accepts it for verification. But nano-002 (store hash) truncates it at 60 chars. The stored hash is silently corrupted. All three nano-tasks pass their individual tests.
3. **The test never actually calls the implementation.** The test imports a mock or an empty stub. GREEN. The implementation file is empty. The user has nothing.

. Anti-Bluff Measures

Every nano-task execution MUST produce:

1. **RED verdict** — the test template actually ran and actually failed before implementation existed. The verdict file carries:

```
{
  "nano_id": "nano-001",
  "polarity": "RED",
  "exit_code": 1,
  "artifact_fingerprint": "sha256:abc123pre",
  "timestamp": "2026-07-24T10:00:00Z",
  "stderr_snippet": "Error: Cannot find module './impl'",
  "evidence_class": "runtime",
  "iterations": 1
}
```

2. **GREEN verdict** — the SAME test ran and passed after implementation. Verdict carries a DIFFERENT artifact fingerprint:

```
{
  "nano_id": "nano-001",
  "polarity": "GREEN",
  "exit_code": 0,
  "artifact_fingerprint": "sha256:def456post",
  "timestamp": "2026-07-24T10:05:00Z",
  "evidence_class": "runtime",
  "iterations": 3
}
```


3. **Class-matched evidence per §11.4.226** — a nano-task whose defect layer is `runtime` (user-visible behavior) MUST close on `runtime`-class evidence. A nano-task that produces a user-visible output (password hash, API response, rendered UI) CANNOT close on source-class evidence (grep for the function name).

4. **Mutation-test the test itself per §1.1** — the meta-test harness introduces a deliberate regression (e.g., truncate the hash to 50 chars) and asserts the GREEN test now FAILs. If the test stays green with the regression, the test is a bluff gate.

5. **Binder-level integration test** — the ONLY test that proves nano-tasks compose correctly. It exercises the full pipeline: validate → hash → store → verify. No nano-task's individual PASS substitutes for the binder's integration PASS.

```go
// anti_bluff/verdict.go
type Verdict struct {
    NanoID            string   `json:"nano_id"`
    Polarity          string   `json:"polarity"`    // RED | GREEN | VERIFY
    ExitCode          int      `json:"exit_code"`
    ArtifactFingerprint string `json:"artifact_fingerprint"`
    Timestamp         string   `json:"timestamp"`
    StderrSnippet     string   `json:"stderr_snippet,omitempty"`
    EvidenceClass     string   `json:"evidence_class"`
    EvidenceFiles     []string `json:"evidence_files"`
    Iterations        int      `json:"iterations"`
}

func ValidateVerdictPair(red, green Verdict, defectLayerEvidenceClass string) error {
    // 1. RED must have failed
    if red.ExitCode == 0 {
        return fmt.Errorf("RED verdict for %s has exit code 0 — "+
            "test did not catch the absent implementation (§11.4.115(F) bluff)", red.NanoID)
    }

    // 2. GREEN must have passed
    if green.ExitCode != 0 {
        return fmt.Errorf("GREEN verdict for %s has exit code %d — implementation is broken",
            green.NanoID, green.ExitCode)
    }

    // 3. Artifact fingerprints MUST differ (proves the fix was deployed)
    if red.ArtifactFingerprint == green.ArtifactFingerprint {
        return fmt.Errorf("RED and GREEN fingerprints are identical for %s — "+
            "the fix was never deployed (§11.4.115(F))", red.NanoID)
    }

    // 4. GREEN evidence class MUST meet the defect layer floor
    if !evidenceClassMeetsFloor(green.EvidenceClass, defectLayerEvidenceClass) {
        return fmt.Errorf("GREEN evidence class %q for %s is below the "+
            "defect layer floor %q (§11.4.226)",
            green.EvidenceClass, green.NanoID, defectLayerEvidenceClass)
    }

    // 5. GREEN must have ≥3 iterations (§11.4.50)
    if green.Iterations < 3 {
        return fmt.Errorf("GREEN verdict for %s has only %d iterations "+
            "(need ≥3 for deterministic consistency)", green.NanoID, green.Iterations)
    }
```

```
    return nil
}

func evidenceClassMeetsFloor(actual, floor string) bool {
    rank := map[string]int{"runtime": 3, "artifact": 2, "source": 1}
    return rank[actual] >= rank[floor]
}
```

. Hardware Optimization

. Threadripper NUMA Awareness

The default target host is an AMD Threadripper 64-core (likely 4 NUMA nodes, 16 cores each). The executor MUST:

1. **Pin workers to NUMA nodes.** Each subagent worker is pinned to a specific NUMA node. Memory for that worker is allocated from the local node. This minimizes cross-node memory latency (which can be 2× local latency).
2. **Topology-aware scheduling.** The executor reads `/sys/devices/system/node/node*/cpulist` to discover NUMA topology. When dispatching parallel nano-tasks, it prefers to allocate workers on the least-loaded NUMA node.
3. **GPU affinity.** CUDA-accelerated nano-tasks (`requires_gpu: true`) are dispatched to workers on the NUMA node closest to the GPU (discovered via `nvidia-smi topo -m`).

```go
// hardware/numa.go
package hardware

import (
    "fmt"
    "os"
    "path/filepath"
    "strconv"
    "strings"
)

type NUMANode struct {
    ID      int
    CPUs    []int
    MemoryGB float64
}

func DiscoverNUMATopology() ([]NUMANode, error) {
    nodeDirs, err := filepath.Glob("/sys/devices/system/node/node[0-9]*")
    if err != nil || len(nodeDirs) == 0 {
        // Non-NUMA system or no sysfs — return single node
        return []NUMANode{{ID: 0, CPUs: allCPUs(), MemoryGB: totalMemoryGB()}}, nil
    }

    var nodes []NUMANode
    for _, dir := range nodeDirs {
        idStr := strings.TrimPrefix(filepath.Base(dir), "node")
        id, _ := strconv.Atoi(idStr)

        // Read CPU list
        cpulist, _ := os.ReadFile(filepath.Join(dir, "cpulist"))
        cpus := parseCPURange(strings.TrimSpace(string(cpulist)))

        // Read memory (meminfo has MemTotal per node)
        meminfo, _ := os.ReadFile(filepath.Join(dir, "meminfo"))
        memKB := parseMemTotal(string(meminfo))

        nodes = append(nodes, NUMANode{
            ID:      id,
            CPUs:    cpus,
            MemoryGB: float64(memKB) / 1024 / 1024,
        })
    }
    return nodes, nil
}
```

```
func parseCPURange(s string) []int {
    var cpus []int
    for _, part := range strings.Split(s, ",") {
        if strings.Contains(part, "-") {
            rangeParts := strings.SplitN(part, "-", 2)
            start, _ := strconv.Atoi(strings.TrimSpace(rangeParts[0]))
            end, _ := strconv.Atoi(strings.TrimSpace(rangeParts[1]))
            for i := start; i <= end; i++ {
                cpus = append(cpus, i)
            }
        } else {
            cpu, _ := strconv.Atoi(strings.TrimSpace(part))
            cpus = append(cpus, cpu)
        }
    }
    return cpus
}
```

. CUDA Detection

```go
// hardware/cuda.go
type GPUInfo struct {
    Index       int
    Name        string
    MemoryMB    int
    NUMANode    int    // Closest NUMA node
    ComputeCap  string
}

func DiscoverGPUs() ([]GPUInfo, error) {
    // Use nvidia-smi to query GPU topology
    cmd := exec.Command("nvidia-smi",
        "--query-gpu=index,name,memory.total,compute_cap",
        "--format=csv,noheader,nounits")
    out, err := cmd.Output()
    if err != nil {
        return nil, fmt.Errorf("nvidia-smi not available: %w", err)
    }

    var gpus []GPUInfo
    for _, line := range strings.Split(strings.TrimSpace(string(out)), "\n") {
        fields := strings.Split(line, ", ")
        if len(fields) < 4 {
            continue
        }
        idx, _ := strconv.Atoi(strings.TrimSpace(fields[0]))
        mem, _ := strconv.Atoi(strings.TrimSpace(fields[2]))

        // Determine NUMA affinity via nvidia-smi topo
        numaNode := discoverGPUNUMANode(idx)

        gpus = append(gpus, GPUInfo{
            Index:      idx,
            Name:       strings.TrimSpace(fields[1]),
            MemoryMB:   mem,
            NUMANode:   numaNode,
            ComputeCap: strings.TrimSpace(fields[3]),
        })
    }
    return gpus, nil
}
```

. KV Cache Sizing

The local LLM's KV cache size determines how many tokens can be processed in a single pass. The executor queries the model's maximum context length and reserves headroom:

```
func computeKVBudget(modelMaxContext int) (systemPrompt, nanoTask, headroom int) {
    // Model max context: e.g., 8192 for Llama-3-8B
    headroom = 512                    // Safety margin
    systemPrompt = 1500                  // Governance preamble + skills
    nanoTask = modelMaxContext - systemPrompt - headroom
    if nanoTask > 512 {
        nanoTask = 512                // Enforce the 512 ceiling
    }
    return
}
```

---

12 1 512

. Token Budget Analysis

. Derivation of the -Token Ceiling

Factor	Tokens	Notes
Model max context (7B class, 4-bit)	8192	Qwen2.5-7B-Instruct, Llama-3-8B
System prompt (governance, skills)	1500	§11.4 preamble + TDD skill + nano-task spec
Safety headroom	512	For unexpected prompt expansion
Remaining budget	6180	
Test template (average)	200	RED test code
Dependency output previews	400	Outputs of 2–3 upstream nano-tasks
Available for implementation	**5580**	But we cap at **512** for three reasons:
		1. Smaller models (3B class) have 4096 context
		2. Budget includes conversation turns (RED→GREEN→fix)
		3. Nano-task should fit in a SINGLE inference pass

. Budget Enforcement Code

```go
// budget/enforce.go
type TokenCounter struct {
    MaxBudget int
}

func (tc *TokenCounter) CountImplementation(implCode string) int {
    // Use the model's tokenizer for accurate counting.
    // For estimation: 1 token ≈ 4 characters (English), ≈ 0.75 words.
    // Production: use tiktoken-go or the model's native tokenizer.

    words := strings.Fields(implCode)
    // Rough estimate: 1.3 tokens per word for code
    estimated := int(float64(len(words)) * 1.3)
    return estimated
}

func (tc *TokenCounter) Validate(nanoID string, implCode string) error {
    count := tc.CountImplementation(implCode)
    if count > tc.MaxBudget {
        return fmt.Errorf(
            "nano-task %s: implementation is %d tokens (budget: %d) — "+
                "MUST be re-decomposed into smaller nano-tasks. "+
                "Consider splitting into %d nano-tasks.",
            nanoID, count, tc.MaxBudget,
            int(math.Ceil(float64(count)/float64(tc.MaxBudget))),
        )
    }
    return nil
}
```

. Re-Decomposition Trigger

When a nano-task exceeds the budget post-implementation, the executor:

1. Flags the nano-task as `OVER_BUDGET`.
2. Records the actual token count.
3. Triggers a targeted re-decomposition: the oversized nano-task is split into 2–3 smaller nano-tasks by the decomposer.
4. The new nano-tasks inherit the existing dependencies and test coverage.

5. The old oversized nano-task is marked `Obsolete` per §11.4.90 with reason `superseded-by-re-decomposition`.

---

13 1

. Code Examples — Decomposer

. Full Decomposer Service (Go)

```go
// service/decomposer_service.go
package service

import (
    "context"
    "encoding/json"
    "fmt"
    "log"
    "os"
    "path/filepath"

    "github.com/HelixDevelopment/helix_nano_bridge/dag"
    "github.com/HelixDevelopment/helix_nano_bridge/decompose"
    "gopkg.in/yaml.v3"
)

type DecomposerService struct {
    LLMClient   LLMClient
    OutputDir   string
    MaxTokens   int
    Logger      *log.Logger
}

type LLMClient interface {
    Complete(ctx context.Context, prompt string, opts CompletionOptions) (string, error)
}

type CompletionOptions struct {
    MaxTokens   int
    Temperature float64
    Model       string
}

// DecomposeSpecKitWorkspace takes a path to a SpecKit tasks.md and produces
// the full nano-task workspace.
func (s *DecomposerService) DecomposeSpecKitWorkspace(
    ctx context.Context, tasksMDPath string,
) (*Workspace, error) {
    // Parse tasks
    tasks, err := decompose.ParseTasksMD(tasksMDPath)
    if err != nil {
        return nil, fmt.Errorf("parse: %w", err)
    }
    s.Logger.Printf("parsed %d tasks from %s", len(tasks), tasksMDPath)
```

```
    ws := NewWorkspace(s.OutputDir)

    for _, task := range tasks {
        binder, err := s.decomposeTask(ctx, task, ws)
        if err != nil {
            return nil, fmt.Errorf("task %s: %w", task.ID, err)
        }
        ws.AddBinder(binder)
    }

    // Cross-cutting pass
    s.extractCrossCutting(ws)

    // Generate binding nano-tasks
    for _, binder := range ws.Binders {
        binder.GenerateBindingNanoTask()
    }

    // Validate DAG
    if _, err := ws.Graph.TopologicalSort(); err != nil {
        return nil, fmt.Errorf("DAG validation: %w", err)
    }

    // Write to disk
    if err := ws.WriteToDisk(s.OutputDir); err != nil {
        return nil, fmt.Errorf("write: %w", err)
    }

    // Write execution plan
    groups, _ := ws.Graph.ParallelGroups()
    plan := map[string]interface{}{
        "total_nano_tasks": len(ws.AllNanoTasks()),
        "total_binders":    len(ws.Binders),
        "groups":           groups,
        "max_parallelism":  maxGroupSize(groups),
    }
    planData, _ := json.MarshalIndent(plan, "", "  ")
    os.WriteFile(filepath.Join(s.OutputDir, "execution-plan.json"), planData, 0644)

    return ws, nil
}

func (s *DecomposerService) decomposeTask(
    ctx context.Context,
    task decompose.Task,
    ws *Workspace,
```

```
) (*Binder, error) {
    binder := NewBinder(task)
    existingIDs := ws.AllNanoTaskIDs()

    // Coarse decomposition via LLM
    coarsePrompt := buildCoarseDecompositionPrompt(task)
    coarseResp, err := s.LLMClient.Complete(ctx, coarsePrompt, CompletionOptions{
        MaxTokens: 2000,
        Model:     "qwen2.5-7b-instruct",
    })
    if err != nil {
        return nil, fmt.Errorf("coarse decomposition: %w", err)
    }

    var coarseTasks []decompose.CoarseSubTask
    if err := json.Unmarshal([]byte(coarseResp), &coarseTasks); err != nil {
        return nil, fmt.Errorf("parse coarse decomposition: %w", err)
    }

    for _, c := range coarseTasks {
        // Nano decomposition via LLM
        nanoPrompt := buildNanoDecompositionPrompt(c, binder.ID, existingIDs, s.MaxTokens)
        nanoResp, err := s.LLMClient.Complete(ctx, nanoPrompt, CompletionOptions{
            MaxTokens: 4000,
            Model:     "qwen2.5-7b-instruct",
        })
        if err != nil {
            return nil, fmt.Errorf("nano decomposition for %s: %w", c.ID, err)
        }

        var nanoSpecs []decompose.NanoTaskSpec
        if err := json.Unmarshal([]byte(nanoResp), &nanoSpecs); err != nil {
            return nil, fmt.Errorf("parse nano decomposition for %s: %w", c.ID, err)
        }

        for _, spec := range nanoSpecs {
            if spec.EstimatedTokens > s.MaxTokens {
                return nil, fmt.Errorf(
                    "nano-task %s exceeds budget: %d > %d tokens",
                    spec.ID, spec.EstimatedTokens, s.MaxTokens,
                )
            }

            nano := NewNanoTask(spec, binder.ID)
            binder.AddNanoTask(nano)
```

```
            node := &dag.Node{
                ID:           nano.ID,
                Dependencies: nano.Dependencies,
            }
            if err := ws.Graph.AddNode(node); err != nil {
                return nil, fmt.Errorf("add node %s: %w", nano.ID, err)
            }

            existingIDs[nano.ID] = struct{}{}
        }
    }

    return binder, nil
}
```

```
    .    TypeScript Decomposer (for in-browser/
small runs)
```

```ts
// decompose/decomposer.ts
import { TaskGraph, NanoTaskNode } from "../dag/graph";
import * as yaml from "js-yaml";

export interface DecomposerOptions {
  maxTokens: number;
  model: string;
  outputDir: string;
}

export class NanoTaskDecomposer {
  constructor(
    private llm: LLMClient,
    private options: DecomposerOptions = { maxTokens: 512, model: "local", outputDir: ".nano-tasks" }
  ) {}

  async decompose(tasksMD: string): Promise<Workspace> {
    const tasks = this.parseTasksMD(tasksMD);
    const ws = new Workspace(this.options.outputDir);

    for (const task of tasks) {
      const binder = await this.decomposeTask(task, ws);
      ws.addBinder(binder);
    }

    this.extractCrossCutting(ws);

    for (const binder of ws.binders) {
      binder.generateBindingNanoTask();
    }

    ws.graph.topologicalSort(); // validates no cycles
    await ws.writeToDisk(this.options.outputDir);
    return ws;
  }

  private parseTasksMD(content: string): SpecKitTask[] {
    const tasks: SpecKitTask[] = [];
    const lines = content.split("\n");
    let current: SpecKitTask | null = null;

    for (const line of lines) {
      const hdrMatch = line.match(/^##\s+(.+)$/);
      if (hdrMatch) {
        if (current) tasks.push(current);
```

```
      current = { id: "", title: hdrMatch[1], description: "", subtasks: [], dependsOn: [] };
      const parts = hdrMatch[1].split(":");
      if (parts.length > 1) {
        current.id = parts[0].trim();
        current.title = parts.slice(1).join(":").trim();
      }
      continue;
    }

    if (!current) continue;

    const subMatch = line.match(/^-\s+\[ \]\s+(.+)$/);
    if (subMatch) {
      current.subtasks.push(subMatch[1].trim());
      continue;
    }

    const depMatch = line.match(/depends on:\s*(.+)$/i);
    if (depMatch) {
      current.dependsOn = depMatch[1].split(",").map(s => s.trim());
      continue;
    }

    current.description += line + "\n";
  }
  if (current) tasks.push(current);
  return tasks;
}

private async decomposeTask(
  task: SpecKitTask,
  ws: Workspace
): Promise<Binder> {
  const binder = new Binder(task);
  const existingIDs = new Set(ws.allNanoTaskIDs());

  // Coarse pass
  const coarsePrompt = this.buildCoarsePrompt(task);
  const coarseResp = await this.llm.complete(coarsePrompt, { maxTokens: 2000 });
  const coarseTasks: CoarseSubTask[] = JSON.parse(coarseResp);

  for (const c of coarseTasks) {
    // Nano pass
    const nanoPrompt = this.buildNanoPrompt(c, binder.id, existingIDs);
    const nanoResp = await this.llm.complete(nanoPrompt, { maxTokens: 4000 });
    const specs: NanoTaskSpec[] = JSON.parse(nanoResp);
```

```
    for (const spec of specs) {
      if (spec.estimatedTokens > this.options.maxTokens) {
        throw new Error(`nano-task ${spec.id} exceeds token budget`);
      }

      const nano = new NanoTask(spec, binder.id);
      binder.addNanoTask(nano);

      const node: NanoTaskNode = {
        id: nano.id,
        dependencies: nano.dependencies,
        status: "pending",
        attempts: 0,
      };
      ws.graph.addNode(node);
      existingIDs.add(nano.id);
    }
  }

  return binder;
 }
}

// ── Prompt Builders ──

function buildCoarsePrompt(task: SpecKitTask): string {
 return `
Decompose this SpecKit task into 4–8 coarse sub-tasks:

TASK: ${task.title}
${task.description}
Subtasks:
${task.subtasks.map(s => `- ${s}`).join("\n")}

Output JSON array of {id, title, description, estimatedNanoTasks, tokenBudget}.
`.trim();
}

function buildNanoPrompt(
 coarse: CoarseSubTask,
 binderId: string,
 existingIDs: Set<string>
): string {
 return `
Decompose this coarse sub-task into 1–5 nano-tasks (≤512 tokens each):
```

```
COARSE: ${coarse.title}
${coarse.description}

Existing nano-task IDs you may depend on: ${[...existingIDs].join(", ") || "none"}
Parent binder: ${binderId}

Each nano-task MUST have: id, title, description, type, inputSchema with example,
outputSchema with example, dependencies[], estimatedTokens, testTemplate (RED-first),
implementationHints[], acceptance[].

Output JSON array only.
`.trim();
}
```

. Code Examples — Full Executor (TypeScript)

```typescript
// executor/executor.ts
import { TaskGraph, NanoTaskNode, NodeStatus } from "../dag/graph";
import * as fs from "fs";
import * as path from "path";
import * as yaml from "js-yaml";

export interface ExecutorOptions {
  maxParallel: number;
  outputDir: string;
  onProgress?: (nodeId: string, status: NodeStatus, message: string) => void;
}

export interface ExecutionResult {
  nodeId: string;
  status: NodeStatus;
  outputPath?: string;
  tokensUsed: number;
  errors: string[];
}

export class NanoTaskExecutor {
  private graph: TaskGraph;
  private results: Map<string, ExecutionResult> = new Map();

  constructor(
    graph: TaskGraph,
    private options: ExecutorOptions = { maxParallel: 4, outputDir: ".nano-output" }
  ) {
    this.graph = graph;
  }

  async run(): Promise<Map<string, ExecutionResult>> {
    const groups = this.graph.parallelGroups();
    this.log(`Starting execution: ${groups.length} groups, ${this.graph.size()} total nano-tasks`);

    for (const group of groups) {
      this.log(`Group: ${group.join(", ")} (${group.length} parallel)`);
      await this.executeGroup(group);
    }

    const incomplete = this.countIncomplete();
    if (incomplete > 0) {
      throw new Error(`${incomplete} nano-tasks did not reach completed status`);
    }
```

```
    this.log("All nano-tasks completed.");
    return this.results;
  }

  private async executeGroup(nodeIds: string[]): Promise<void> {
    // Limit parallelism
    const semaphore = new Semaphore(this.options.maxParallel);
    const promises = nodeIds.map(id => semaphore.run(() => this.executeNanoTask(id)));
    await Promise.all(promises);
  }

  private async executeNanoTask(nodeId: string): Promise<void> {
    const node = this.graph.getNode(nodeId);
    if (!node) throw new Error(`Node ${nodeId} not found`);

    this.updateStatus(nodeId, "running", "execution started");

    try {
      // Load dependency outputs
      const depOutputs = await this.loadDependencyOutputs(node);

      // ── RED phase ──
      const redResult = await this.runTest(node, depOutputs, "RED");
      if (redResult.exitCode === 0) {
        throw new Error(
          `RED test PASSED for ${nodeId} — test is not catching absent ` +
          `implementation (bluff gate, §11.4.115(F))`
        );
      }
      this.log(`  [${nodeId}] RED: FAIL (expected)`);

      // ── IMPLEMENT phase ──
      const implResult = await this.dispatchImplementation(node, depOutputs);
      this.log(`  [${nodeId}] IMPL: ${implResult.tokensUsed} tokens`);

      if (implResult.tokensUsed > 512) {
        this.log(`  [${nodeId}] WARNING: ${implResult.tokensUsed} tokens (budget: 512) — flag for re-
              decomposition`);
      }

      // ── GREEN phase ──
      const greenResult = await this.runTest(node, depOutputs, "GREEN");
      if (greenResult.exitCode !== 0) {
        throw new Error(
          `GREEN test FAILED for ${nodeId}: ${greenResult.stderr}`
        );
```

```
    }
    this.log(`  [${nodeId}] GREEN: PASS`);

    // ── VERIFY phase (3× determinism, §11.4.50) ──
    for (let i = 0; i < 3; i++) {
      const verifyResult = await this.runTest(node, depOutputs, "VERIFY");
      if (verifyResult.exitCode !== 0) {
        throw new Error(
          `VERIFY iteration ${i + 1}/3 FAILED for ${nodeId} — ` +
          `non-deterministic behavior (§11.4.50 violation)`
        );
      }
    }

    // Write verdicts
    await this.writeVerdicts(nodeId, redResult, greenResult);

    this.updateStatus(nodeId, "completed", "all phases passed");
  } catch (err) {
    this.updateStatus(nodeId, "failed", String(err));
    throw err;
  }
}

private async runTest(
  node: NanoTaskNode,
  depOutputs: Record<string, unknown>,
  mode: "RED" | "GREEN" | "VERIFY"
): Promise<TestResult> {
  // In production, this runs the test in a sandboxed environment
  // (container or isolated process).  For the specification, we
  // show the interface.
  const testRunner = new SandboxedTestRunner(node, depOutputs, mode);
  return testRunner.execute();
}

private async dispatchImplementation(
  node: NanoTaskNode,
  depOutputs: Record<string, unknown>
): Promise<ImplResult> {
  // Dispatch to the LLM subagent.  In production, this calls the
  // Superpowers bridge or the local LLM directly.
  const subagent = new SubagentDispatcher(node, depOutputs);
  return subagent.dispatch();
}
```

```
  private async writeVerdicts(
    nodeId: string,
    red: TestResult,
    green: TestResult
  ): Promise<void> {
    const dir = path.join(this.options.outputDir, "verdicts", nodeId);
    fs.mkdirSync(dir, { recursive: true });

    const redVerdict = {
      nano_id: nodeId,
      polarity: "RED",
      exit_code: red.exitCode,
      artifact_fingerprint: red.fingerprint,
      evidence_class: "runtime",
      timestamp: new Date().toISOString(),
    };
    fs.writeFileSync(
      path.join(dir, "RED.verdict.json"),
      JSON.stringify(redVerdict, null, 2)
    );

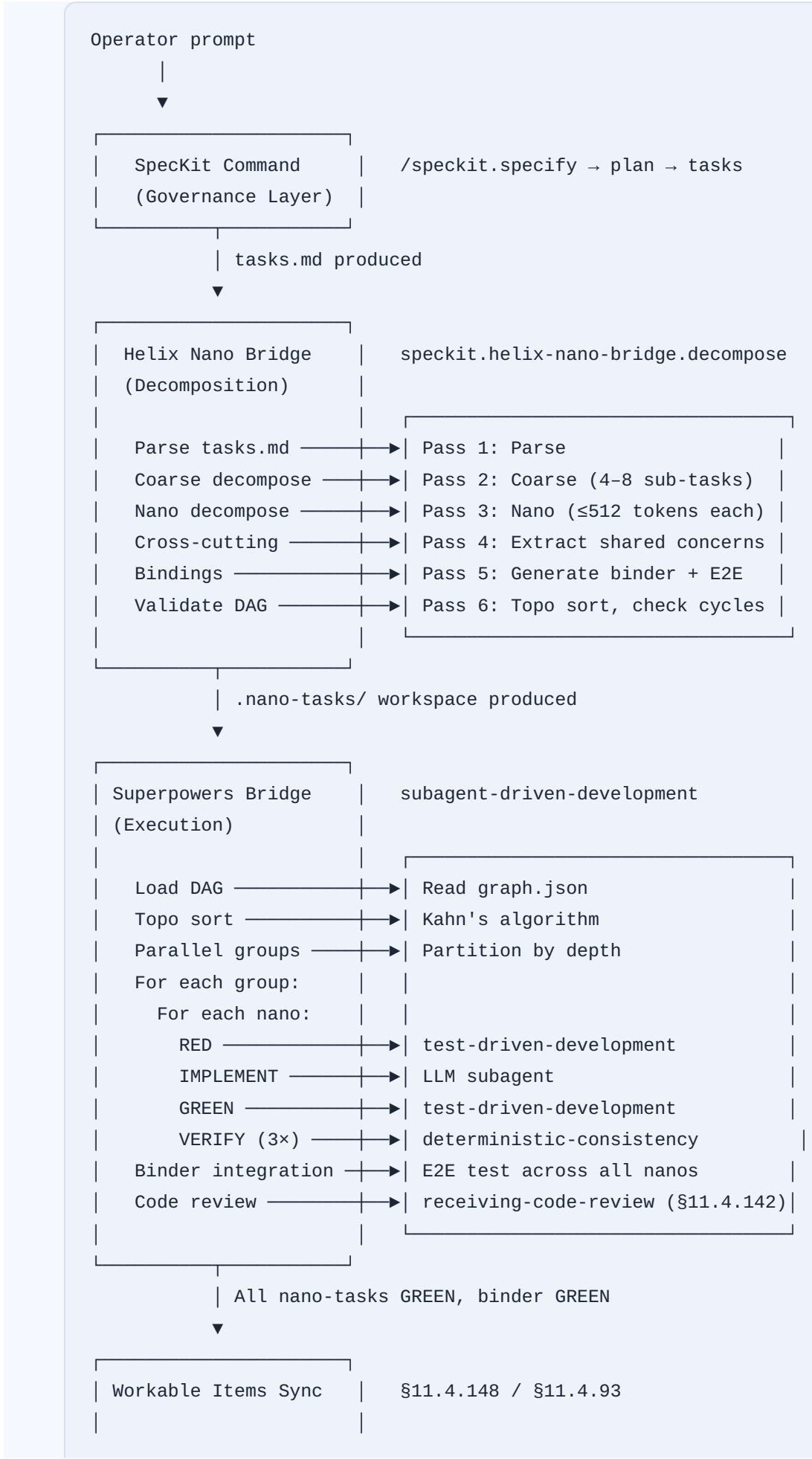
    const greenVerdict = {
      nano_id: nodeId,
      polarity: "GREEN",
      exit_code: green.exitCode,
      artifact_fingerprint: green.fingerprint,
      evidence_class: "runtime",
      iterations: 3,
      timestamp: new Date().toISOString(),
    };
    fs.writeFileSync(
      path.join(dir, "GREEN.verdict.json"),
      JSON.stringify(greenVerdict, null, 2)
    );

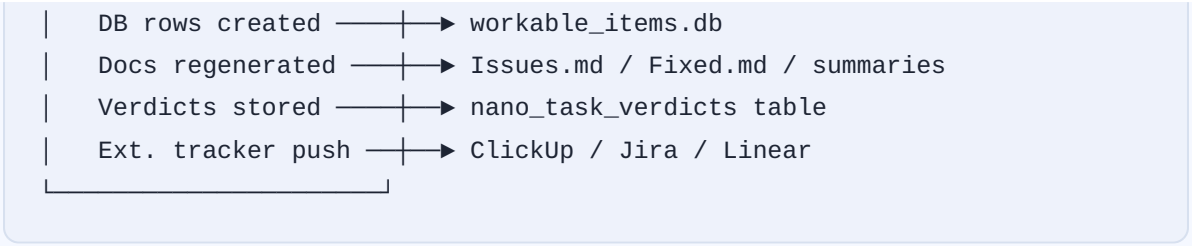
    this.results.set(nodeId, {
      nodeId,
      status: "completed",
      outputPath: dir,
      tokensUsed: 0,
      errors: [],
    });
  }

  private updateStatus(nodeId: string, status: NodeStatus, message: string): void {
    const node = this.graph.getNode(nodeId);
```

```
    if (node) {
      node.status = status;
      this.options.onProgress?.(nodeId, status, message);
    }
  }
}
```

. Integration Flow Summary


Operator prompt
SpecKit Command
(Governance Layer)
/speckit.specify → plan → tasks
tasks.md produced
Helix Nano Bridge
(Decomposition)
speckit.helix-nano-bridge.decompose
Parse tasks.md
Coarse decompose
Nano decompose
Cross-cutting
Bindings
Validate DAG
Pass 1: Parse
Pass 2: Coarse (4–8 sub-tasks)
Pass 3: Nano (≤512 tokens each)
Pass 4: Extract shared concerns
Pass 5: Generate binder + E2E
Pass 6: Topo sort, check cycles
.nano-tasks/ workspace produced
Superpowers Bridge
(Execution)
subagent-driven-development
Load DAG
Topo sort
Parallel groups
For each group:
For each nano:
RED
IMPLEMENT
GREEN
VERIFY (3×)
Binder integration
Code review
Read graph.json
Kahn's algorithm
Partition by depth
test-driven-development
LLM subagent
test-driven-development
deterministic-consistency
E2E test across all nanos
receiving-code-review (§11.4.142)
All nano-tasks GREEN, binder GREEN
Workable Items Sync
§11.4.148 / §11.4.93


DB rows created
workable_items.db
Docs regenerated
Issues.md / Fixed.md / summaries
Verdicts stored
nano_task_verdicts table
Ext. tracker push
ClickUp / Jira / Linear

. Workspace File Structure

```
.nano-tasks/
│
├── workspace.yaml                  # Workspace manifest (version, model,
host info)
│
├── binders/
│   ├── binder-auth.yaml            # One binder per SpecKit task
│   ├── binder-session.yaml
│   └── ...
│
├── nanos/
│   ├── nano-000.yaml               # One YAML per nano-task (the
canonical spec)
│   ├── nano-001.yaml
│   ├── nano-002.yaml
│   └── ...
│
├── graph.json                      # Serialized DAG (nodes + edges)
├── execution-plan.json             # Topologically-sorted parallel groups
│
├── src/                            # Generated implementation code
│   ├── auth/
│   │   ├── validate_password.ts
│   │   ├── hash_password.ts
│   │   ├── store_hash.ts
│   │   └── verify_hash.ts
│   └── session/
│       ├── create_session.ts
│       └── ...
│
├── tests/                          # Generated test code
│   ├── auth/
│   │   ├── validate_password.test.ts
│   │   ├── hash_password.test.ts
│   │   └── ...
│   └── ...
│
├── verdicts/                       # Captured evidence per §11.4.5 /
§11.4.69
│   ├── nano-000/
│   │   ├── RED.verdict.json
│   │   ├── GREEN.verdict.json
│   │   └── VERIFY.verdict.json
│   └── ...
│
```

```
├── qa-results/                    # Full QA transcripts per §11.4.83
│   └── <run-id>/
│       └── ...
│
├── checkpoints/                   # Resumption state
│   └── latest.json
│
└── .nano-task.db                  # SQLite workable-items SSoT
(§11.4.93)
```

This specification defines every component of the Nano-Task Decomposition & Execution Engine at implementation depth — from the YAML schema that governs every nano-task, through the DAG engine's topological sort and parallel grouping algorithms (in both Go and TypeScript), the multi-pass decomposition pipeline, the TDD-driven executor, to the anti-bluff verification that proves every nano-task genuinely works per the §11.4 covenant. Implementations MUST follow this specification exactly; any deviation that weakens a constraint (§11.4.6 no-guessing, §11.4.115 polarity switch, §11.4.50 deterministic consistency, the 512-token ceiling, or the evidence-class floor) is a release blocker.